CST8918 Complete Study Guide

DevOps and Infrastructure as Code | Modules 1-10

This guide combines the course lesson notes with expanded explanations and practical examples. It covers the teaching material from Modules 1-10; quizzes, labs, assignments, the midterm, and the final exam are not attempted.

How to use this guide

Read the expanded explanation at the start of each module to understand the big ideas. Then use the lesson detail that follows for terminology, comparisons, diagrams, and examples from the college notes. Module 1-6 material supports midterm study; Module 6-10 material supports final exam study.

Contents

Module 1: DevOps foundations

Module 2: Infrastructure as Code

Module 3: Version control for infrastructure

Module 4: Continuous integration and delivery

Module 5: Terraform: defining infrastructure

Module 6: Terraform: testing infrastructure code

Module 7: Ansible: configuration and orchestration

Module 8: Ansible: execution and YAML

Module 9: Azure Bicep: resource definitions

Module 10: Azure Bicep: authoring and deployment

Module 1: DevOps foundations

DevOps is a way to deliver software through shared responsibility, frequent feedback, and repeatable automation. Development and operations still have different expertise, but they work on one delivery system and measure the health of the same service.

Expanded explanation

People and culture

The people pillar means that developers, operations staff, testers, and security specialists communicate early and own outcomes together. Culture makes that collaboration routine: small changes, blameless incident review, and rapid feedback replace a handoff in which one team throws work to another.

Automation and measurement

Automation removes repetitive work such as building, testing, packaging, deploying, and configuring environments. Measurement tells the team whether the automation improves outcomes. Useful signals include build failures, deployment frequency, change failure rate, recovery time, and service behavior after release.

The life cycle as a feedback loop

Plan, code, build, test, release, deploy, operate, and monitor are connected stages. Monitoring is not the end: incidents and user feedback return to planning. A small change moving around this loop is easier to understand and repair than a large release assembled over months.

CI, delivery, and deployment

Continuous integration merges small code changes often and checks them with automated builds and tests. Continuous delivery keeps a tested release ready for a deliberate production decision. Continuous deployment goes one step further and automatically releases every change that passes the required gates.

How DevOps extends Agile

Agile shortens the feedback cycle for product development. DevOps carries the same idea through release and operation, so a feature is not truly done merely because its code was written. The team also needs a dependable way to deploy it and see how it behaves.

Practical example

A developer changes a web application. The change is reviewed and merged; a pipeline builds the application, runs tests, and packages a versioned artifact. The team deploys that same artifact to a staging environment, then production. Monitoring reveals an error spike, so the team rolls back or fixes the change and records what to improve in the pipeline.

Key point

Think of DevOps as one continuous learning system: people coordinate the work, automation makes it repeatable, and measurements show whether users actually benefit.

College lesson notes

Introduction

Module 1: DevOps

Welcome to Module 1 of our comprehensive course on DevOps and Infrastructure as Code! Software development has come a long way, evolving from traditional methodologies that often led to siloed teams, slow-release cycles, and manual processes. Enter DevOps - a dynamic cultural and technical movement that challenges these conventions and brings about a paradigm shift in how teams collaborate, deliver software, and manage infrastructure. At the heart of DevOps lies the concept of Infrastructure as Code (IaC), a practice that treats infrastructure provisioning and management as code, allowing for automation, version control, and consistency.

In this course, you will learn about all traditional infrastructure elements of the cloud computing model, such as servers, networks and firewalls, which are virtualized. These components are defined in code and can then be deployed or decommissioned on-demand. Without systematic controls and policy enforcement mechanisms, this flexibility can result in inconsistent performance and wasted resources. Additionally, in this course you will employ industry-standard tools to manage both application code and the virtualized deployment environment. Industry best practice recommendations guide lab scenarios for automating the continuous integration and continuous deployment (CI/CD) of application code and automating infrastructure management tasks such as auto-scaling, container orchestration, and data extract-load-transform (ETL) tasks.

As we begin module 1, you will embark on an exciting journey into the world of modern software development practices that are revolutionizing the way applications are built, deployed, and maintained.

Learning Objectives

By the end of this module, you should be able to complete the following:

- Explain DevOps and its Core Principles.
- Trace the Evolution of DevOps.
- Recognize DevOps as an Extension of Agile.
- Identify the Pillars of DevOps.
- Navigate the DevOps Life Cycle.
- Interpret the Essentials of CI/CD Pipelines.

Key Terms and Concepts

Key Term or Concept	Definition
CI/CD	Continuous Integration and Continuous Delivery (or Continuous Deployment).
IaC	Infrastructure as Code

What is DevOps?

Definition

DevOps is a transformative approach that combines development (Dev) and IT operations (Ops) to enhance collaboration, streamline processes, and deliver high-quality software at a faster pace. It emerged as a response to the limitations of traditional development methodologies. DevOps emphasizes breaking down the barriers between development and operations teams, enabling them to work together seamlessly. By fostering a culture of shared responsibility, automation, and continuous improvement, DevOps aims to achieve faster releases, quicker bug fixes, and more reliable software.

Evolution of DevOps

Before DevOps, traditional software development methodologies like Waterfall resulted in slow, infrequent releases and often lacked effective communication between development and operations teams. DevOps evolved as a response to these challenges, aiming to address the inefficiencies by introducing automation, collaboration, and continuous integration. The movement gained traction as organizations realized the need to deliver software more frequently, respond faster to market changes, and maintain higher quality standards.

The WallThe Wall of confusion and the Origin of DevOps - Andrew Shafer

Dev Team: Occupied with planning, design, development and testing. Product and innovation focus. Goal to create and provide new features.

Ops Team: Involved with product release configurations and monitoring Services focus.Goal to assure the system’s stability.

The Blame Game: Misalign interest. Dev team motivated by speed releases. Ops team motivated by stability. Whose fault is it when a problem arises? Send the problem over the wall.

Pillars of DevOps

The four pillars of DevOps are foundational to successful implementation. The four pillars are as follows:

- People
- Culture
- Automation; and
- Measurement

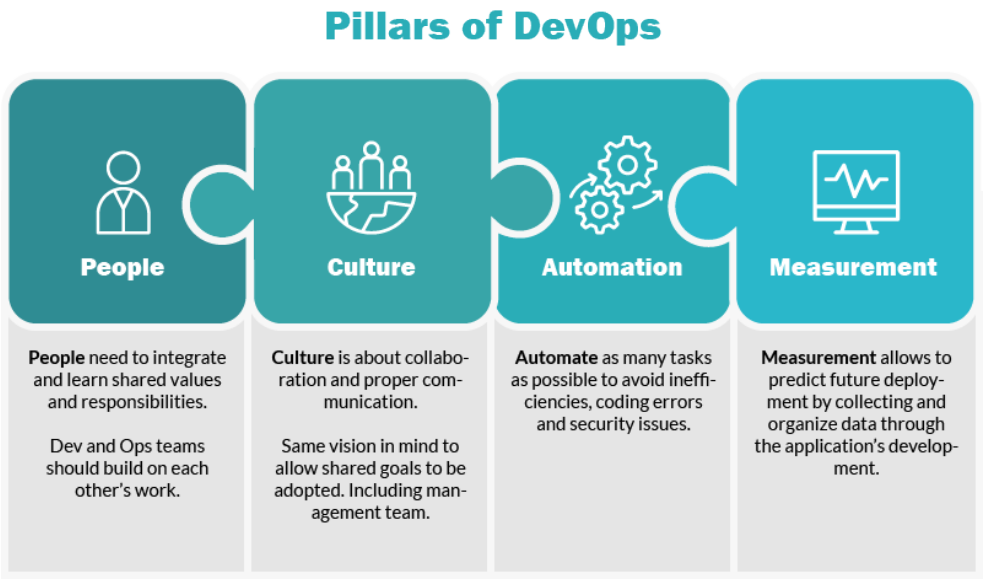
Let’s examine each of these pillars in greater detail.

PeopleKnowledge sharing, and collaboration are vital for a successful DevOps culture. Teams share best practices, tools, and experiences to drive innovation and learning.

CultureEncourages collaboration, shared responsibility, and open communication across teams. It promotes the idea that developers and operations staff should collaborate closely to achieve common goals.

AutomationInvolves automating manual processes, from code compilation and testing to deployment and infrastructure provisioning. Automation reduces human errors, accelerates processes, and enables frequent releases.

Measurement: Collecting data and metrics allows teams to track performance, identify bottlenecks, and make informed decisions for continuous improvement.



People and Culture in DevOpsPeople and culture are integral to DevOps success. Encourage a culture of trust, where teams take ownership of their work. "You build it, you run it" means that developers are responsible for the software they create, even in production. This mindset fosters accountability and a sense of pride in delivering quality code. Communication and empathy between teams ensure that everyone understands the end goals and works collaboratively.

Organizational Cultures in DevOpsTraditional hierarchical cultures hinder collaboration and agility. DevOps requires a shift toward more agile, flat, and collaborative structures. Implement cross-functional teams that include developers, operations, quality assurance, and other stakeholders. Encourage an environment where experimentation, learning from failures, and continuous improvement are valued.

DevOps Life Cycle

The DevOps life cycle encompasses several phases:

- Planning: Teams define requirements, prioritize tasks, and set goals for the development cycle.
- Development: Developers write, and test code based on the requirements.
- Testing: Automated testing is performed to ensure code quality and identify defects early.
- Deployment: Automated deployment processes push code to production-like environments, ensuring consistency and reducing manual errors.
- Monitoring: Continuous monitoring helps detect issues and performance bottlenecks in real-time.
- Feedback: Collect feedback from users and stakeholders to inform future development iterations.

CI/CD Pipeline

The CI/CD pipeline is the backbone of DevOps. It automates the process of integrating code changes, running tests, and deploying to production.

Continuous Integration (CI)

Think of CI as a team of chefs working together in a kitchen to cook a delicious meal. Each chef prepares a specific part of the dish - one for the main course, another for sides, and another for dessert. As they finish their parts, they bring them together on the same table. This way, they're not waiting until the entire meal is ready before they start coordinating. They're integrating their work continuously as they go.

In the world of software development, CI is like these chefs collaborating. Developers write code for a software project, just like chefs prepare different parts of a meal. Instead of waiting until the very end to combine everything, CI tools automatically and frequently integrate the code changes from all the developers into a shared place. This way, any issues or conflicts can be spotted and resolved early on, ensuring that the overall project remains stable and functional.

Example: Imagine a team of programmers working on a mobile app. They use a CI tool that automatically compiles and tests their code every time they make changes. As soon as one developer finishes coding a new feature, the CI tool checks it to make sure it doesn't break anything else in the app. If everything is okay, the tool signals that the code can be safely integrated into the main app's codebase. This way, the team stays updated on each other's progress and avoids last-minute surprises.

Continuous Delivery (CD)

Now, let's take our kitchen analogy a step further. After the chefs have prepared and integrated their individual parts, they place the complete meal on the serving cart. The meal is fully cooked, presented nicely, and ready to be served to the diners whenever they're ready to enjoy it.

In software development, Continuous Delivery is like having your software in that ready-to-serve state. After the code passes through CI and is integrated, the CD process automatically packages the software and prepares it for release. The software is tested, validated, and set up so that it can be released to users at any time with confidence.

Example: Following the mobile app scenario, let's say the code successfully passes through the CI process. Now, the Continuous Delivery pipeline takes over. It compiles the app, runs additional tests, and even deploys it to a staging environment that's almost identical to the real thing. This lets the team ensure that everything works smoothly before they decide to release it to the public. If all looks good, they can press a button to deploy the new version to the app store, making it available for users to download and enjoy. In simple terms, Continuous Integration and Continuous Delivery are like chefs working together to cook a meal in an organized way and then presenting it beautifully for diners to enjoy. Similarly, developers work together to integrate code changes smoothly and deliver software that's well-prepared and ready for users.

Continuous Integration (CI):	Continuous Delivery (CD):
Development practice that involves frequently merging code changes into a central repository and automatically building and testing the code.	Development practice that involves automatically deploying software changes to production after they have passed various stages of testing.

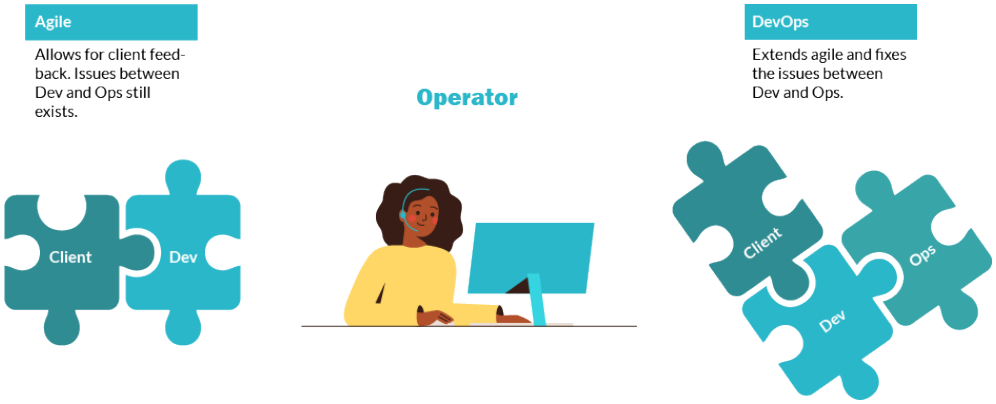
DevOps and CI/CD

DevOps and CI/CD work hand in hand to achieve the common goals of faster delivery, improved quality, and enhanced collaboration:

- Automation and Efficiency: DevOps emphasizes automation throughout the software development lifecycle. CI/CD pipelines automate tasks like code integration, testing, and deployment, which aligns perfectly with the DevOps philosophy of automating manual and repetitive processes.
- Collaboration: DevOps fosters collaboration between development and operations teams, ensuring that they work together to achieve common goals. CI/CD pipelines provide a centralized and automated platform for teams to integrate, test, and deliver code changes, promoting collaboration and reducing conflicts.
- Continuous Feedback: Both DevOps and CI/CD promote the importance of continuous feedback loops. DevOps encourages ongoing communication between teams to improve processes, while CI/CD pipelines provide immediate feedback on the quality of code changes through automated testing.
- Rapid and Reliable Delivery: DevOps seeks to deliver software faster and more reliably. CI/CD pipelines enable this by automating the deployment process, allowing teams to release new features and fixes with confidence and speed.
- Culture of Ownership: DevOps culture encourages developers to take ownership of their code in production. CI/CD supports this by allowing developers to see their code changes integrated, tested, and deployed rapidly, reinforcing the "You build it, you run it" principle.

In essence, DevOps provides the cultural and organizational foundation for collaboration, while CI/CD provides the technical practices and tools to implement automation and continuous delivery. Together, they create an environment where software development becomes a seamless, efficient, and collaborative process, ultimately resulting in faster innovation and higher-quality software releases.

DevOps an Extension of Agile



Agile: Allows for client feedback. Issues between Dev and Ops still exists.

DevOps: Extends agile and fixes the issues between Dev and Ops.

DevOps builds upon Agile principles by extending the concept of iterative development beyond coding to include deployment and operations. Both DevOps and Agile prioritize customer feedback, adaptive planning, and incremental progress. However, DevOps emphasizes the continuous integration of code, automated testing, and continuous delivery, ensuring that software changes can be quickly and reliably deployed to production. This integration of development and operations aligns with Agile's emphasis on collaboration and customer value.

Summary

Conclusion

In this first module, we explored the foundational principles of DevOps and its close relationship with Continuous Integration (CI) and Continuous Delivery (CD). DevOps bridges the gap between development and operations, fostering collaboration, automation, and a culture of continuous improvement. CI/CD complements DevOps by automating code integration, testing, and deployment, resulting in faster, more reliable software delivery. By understanding these concepts, you're equipped to embark on a journey that transforms how software is developed, delivered, and maintained, setting the stage for the exciting exploration of Infrastructure as Code in the next module.

References

- Amazon Web Services. 2023. "What is DevOps?" [Webpage] Retrieved from https://aws.amazon.com/devops/what-is-devops/
- Atlassian. 2023. "DevOps: What it is and why it matters" [webpage] Retrieved from https://www.atlassian.com/devops
- Microsoft. 2023. “What is DevOps?” [Webpage] Retrieved from https://learn.microsoft.com/en-us/devops/what-is-devops

---

Module 2: Infrastructure as Code

Infrastructure as Code (IaC) describes infrastructure in files that can be reviewed, versioned, tested, and applied repeatedly. A server, network, firewall rule, or cloud service becomes part of a documented configuration rather than a collection of manual console clicks.

Expanded explanation

Desired state and repeatability

An IaC file describes either the end state a system should reach or the steps that should run to reach it. Running a well-designed configuration again should leave a correctly configured resource alone. That repeatability helps create matching development, test, and production environments.

Declarative versus imperative

Declarative code says what resources should exist and lets the tool determine the change sequence. Imperative code says how to perform the changes step by step. Both can be useful, but a declarative resource definition is often easier to compare with the current environment and to reapply safely.

Benefits and limits

IaC can reduce manual mistakes, speed up provisioning, and make changes reviewable. It does not automatically make a poor design safe. An incorrect template can reproduce the same mistake everywhere, so review, testing, least privilege, and controlled rollout remain necessary.

Drift and configuration control

Drift occurs when the actual environment no longer matches the recorded configuration, often because someone changed a setting manually. The team should detect drift, decide whether the manual change was valid, and update code or restore the declared state. Configuration files should be the shared record of intent.

Different tool roles

Provisioning tools create and update resources. Configuration-management tools install packages and set up systems. Orchestration coordinates several tasks or services. A real workflow may use more than one tool, with each responsible for a clear part of the environment.

Practical example

Suppose an application needs a virtual network, two application servers, and a database. An IaC configuration defines those resources and their relationships. A reviewer can inspect the proposed change before it runs; a deployment creates the same topology in another environment by changing input values such as names and sizes rather than repeating console steps.

Key point

IaC turns infrastructure changes into reviewable, repeatable work. The code is most valuable when it is paired with testing, version control, and an agreed process for handling drift.

College lesson notes

Introduction

Module 2: Infrastructure as Code (IaC)

Welcome to the second module of our course on DevOps and Infrastructure as Code (IaC)! In this module, we will dive into the fascinating world of Infrastructure as Code, commonly referred to as IaC. IaC is a ground-breaking approach that revolutionizes how we deploy and manage IT infrastructure. By treating infrastructure as if it were software code, IaC brings automation, consistency, and efficiency to the world of operations, aligning perfectly with the DevOps principles we explored in the first module. By the end of this module, you'll have a solid understanding of IaC's core principles, its significance in modern DevOps practices, and the tools that enable you to harness its power effectively.

Learning Objectives

By the end of this module, you should be able to complete the following:

- Explain the fundamental concept of Infrastructure as Code (IaC).
- Recognize the advantages and potential pitfalls of IaC.
- Outline the key tools used in IaC, including Terraform, Ansible, and Bicep.
- Explain how IaC is applied in DevOps practices.

Key Terms and Concepts

Key Term or Concept	Definition
CI/CD	Continuous Integration and Continuous Delivery (or Continuous Deployment).

Key Term or Concept	Definition
IaC	Infrastructure as Code

What is IaC?

Infrastructure as Code (IaC) is a transformative approach to managing and provisioning IT infrastructure using code and automation. Instead of manually configuring servers, networks, and other resources, IaC treats infrastructure as software. This means you define your infrastructure using code, just like you would write software applications. IaC allows you to describe your desired infrastructure configuration in a declarative manner, specifying how various resources should be set up, configured, and connected.

With IaC, you no longer need to rely on manual processes, which are prone to errors and inconsistencies. Instead, you can use version-controlled code to define, deploy, and manage your infrastructure. This approach streamlines provisioning, enhances consistency, and brings the benefits of automation to infrastructure management.

Understanding the Fundamental Concept of Infrastructure as Code (IaC):

Imagine you're building a web application that requires multiple servers, a database, and a load balancer. Instead of manually setting up each server, configuring the database, and connecting them, you write code that defines these resources and their relationships. This code becomes your infrastructure definition. When you want to deploy this setup, you run the code, and the infrastructure is automatically created according to your specifications. If you need to make changes, you update the code and run it again. This way, your infrastructure becomes as adaptable and repeatable as your software code.

Advantages and Potential Pitfalls of IaC

As with most things, there are advantages and disadvantages. Outlined below are some of the more common ones.

Advantages of IaC

- Automation: IaC enables automated provisioning and configuration, reducing manual work and human error.
- Consistency: Code-based definitions ensure uniformity across different environments, reducing configuration drift.
- Version Control: Infrastructure code can be stored in version control systems, enabling collaboration, and tracking changes.
- Reproducibility: You can recreate identical environments at any time, facilitating testing and development.
- Scalability: Scaling up or down becomes more manageable as infrastructure is defined in code.
- Documentation: The code serves as self-documenting infrastructure, making it easier to understand and troubleshoot.

Potential Pitfalls of IaC

- Learning Curve: Teams may need to learn new tools and practices to effectively implement IaC.
- Complexity: Overly complex infrastructure code can become hard to manage and debug.
- Initial Investment: IaC requires upfront effort to set up infrastructure definitions.
- Tool Dependencies: Relying heavily on specific IaC tools may lead to vendor lock-in.

Key Tools Used in IaC

A wide range of Infrastructure as Code tools are available today to help IT Managers address some of the following infrastructure management tasks.

- Provisioning
- Deployment
- Configuration
- Orchestration

In the past, installing, configuring, and updating programs for Cloud servers was a manual task for IT managers. Similarly, the network teams had to manually store and manage the configuration data. This was not only a time-consuming and tedious process, but also called for the collaboration of several IT personnel. The most pressing problem was scalability because

administrators found it challenging to bring up new servers quickly enough to keep up with the speed and scope of continuously evolving business operations. Furthermore, hiring and managing a team increased costs significantly.

It is important to be aware that some IaC tools are involved in the infrastructure setup, while others manage the infrastructure or the applications in your environment. For the purpose of this course, we will discuss some of the popular tools below.

Terraform Terraform is a widely-used open-source tool that provides Infrastructure as Code capabilities. It allows you to define, provision, and manage infrastructure resources across various cloud providers and even on-premises environments. Terraform uses a declarative approach, where you define your desired infrastructure configuration in HashiCorp Configuration Language (HCL) or JSON. It then creates and manages resources while ensuring that your infrastructure remains in the desired state.



Ansible:



Ansible is a powerful configuration management tool that automates software provisioning, configuration, and application deployment. Unlike some other IaC tools, Ansible uses an imperative approach, where you define the specific tasks that need to be executed. Ansible Playbooks, written in YAML, describe the desired configuration and steps to achieve it. It works not only for infrastructure provisioning but also for software deployment and orchestration.

Bicep:



Bicep is a domain-specific language (DSL) developed by Microsoft for creating Azure Resource Manager (ARM) templates. It's designed to make ARM template creation more readable and efficient. Bicep files are transpiled into ARM JSON templates, providing a more streamlined way to define Azure infrastructure resources.

AWS CloudFormation:



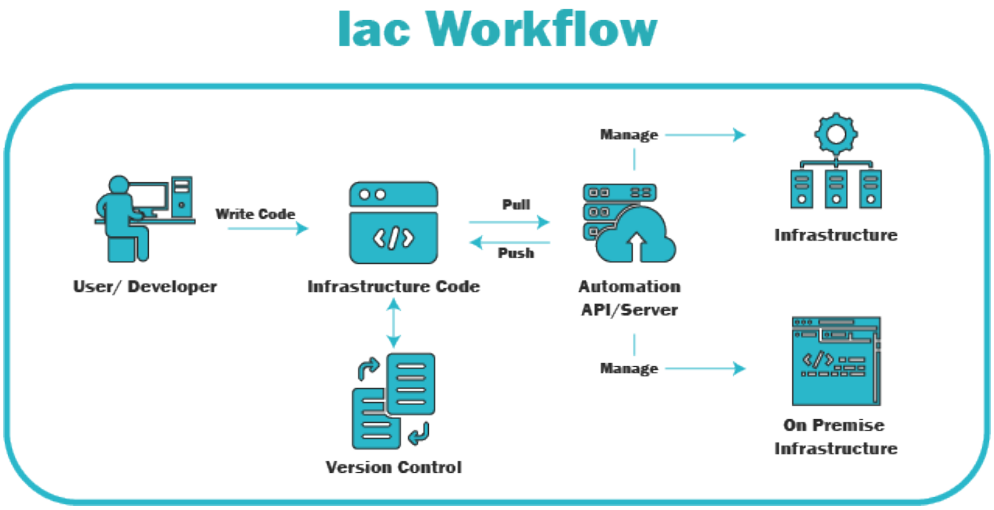
AWS CloudFormation is a service provided by Amazon Web Services (AWS) that enables you to provision and manage resources in AWS using Infrastructure as Code. Similar to Terraform and Bicep, CloudFormation allows you to define your infrastructure using a declarative approach. You create templates using JSON or YAML to specify the resources you need, their configurations, and relationships. CloudFormation then takes care of creating, updating, and deleting resources while maintaining their desired state.

ComparisonLet’s compare each of these tools even more.

Terraform	Ansible	Bicep	AWS CloudFormation
Supports provisioning and managing resources across different cloud providers and on-premises environments.	Focuses on configuration management, automating software provisioning, configuration, and application deployment.	Specializes in creating Azure Resource Manager (ARM) templates for Azure resources.	Specifically designed for provisioning and managing resources in Amazon Web Services (AWS) cloud environments.

Each of these tools caters to different cloud platforms and use cases. Depending on your cloud provider and preferences, you can choose the tool that best aligns with your needs and infrastructure environment.

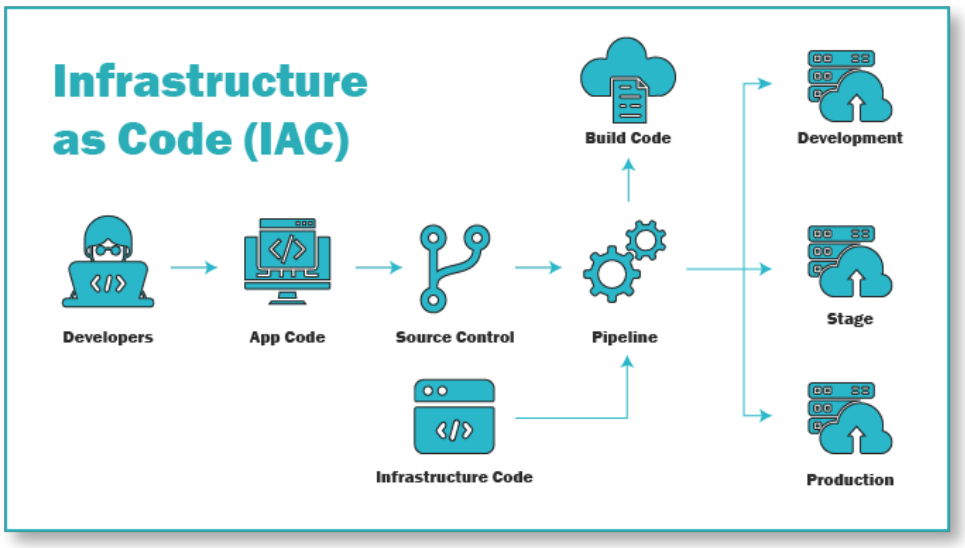
Example Let's use Terraform as an example for a second. If you want to set up a virtual machine on a cloud provider, you can use Terraform code to define the VM, its specifications, and any additional resources it requires, like storage and networking. When you run the Terraform code, it communicates with the cloud provider's API and creates the VM based on your specifications. This approach is much more efficient than manually clicking through the provider's dashboard to set up each component. A general workflow of how IaC works looks something like this.



Understanding How IaC Is Applied in DevOps Practices:

IaC plays a crucial role in DevOps by enabling consistent, automated, and version-controlled infrastructure deployment. In a DevOps workflow, IaC is integrated into the continuous integration and continuous delivery (CI/CD) pipelines. Changes to infrastructure code are subjected to the same testing and review processes as application code, ensuring that infrastructure changes are reliable and safe. This integration allows for faster and more consistent deployments, reduced errors, and improved collaboration between development and operations teams.

Below is a diagram showing the use of IaC in a typical CI/CD pipeline.



By incorporating IaC into DevOps practices, organizations can achieve faster software delivery, greater reliability, and increased flexibility in responding to changing business needs.

Example Imagine you're part of a DevOps team working on a web application. When you make changes to the application code, you also need to ensure that the required infrastructure is updated accordingly. With IaC, you integrate infrastructure code into your CI/CD pipeline. Whenever you push changes to the application's repository, the pipeline automatically triggers, testing and deploying both the application code and the infrastructure code. This ensures that any updates to the application are reliably supported by the correct infrastructure configuration.

Declarative IaC Vs Imperative IaC:

Declarative and imperative are two different approaches to writing Infrastructure as Code (IaC) scripts that define and manage IT infrastructure. Let's break down what each approach means:

1. Declarative IaC

In a declarative approach, you describe the desired end state of your infrastructure without specifying the exact steps to get there. You define what you want the infrastructure to look like, and the IaC tool figures out how to achieve that state. Declarative IaC focuses on the "what" rather than the "how." It emphasizes the outcome, allowing the tool to handle the implementation details.

Here's an example. In declarative IaC, you might specify that you want three virtual machines with specific configurations, networks, and security settings. The IaC tool takes care of provisioning those resources according to your specifications.

2. Imperative IaC

In an imperative approach, you provide step-by-step instructions for how to achieve the desired infrastructure configuration. You define explicit actions and sequences, essentially telling the tool what actions to take in order to create or modify resources. Imperative IaC is more focused on the specific actions and processes needed to reach the desired state.

An example of imperative IaC, is when you might specify commands to create individual resources, like creating a virtual machine, attaching storage, and configuring networking.

ComparisonLet's take a moment and distinguish between the two approaches.

Declarative	Imperative
Focuses on the desired end state. The IaC tool figures out the steps to reach that state. It's like saying "I want a cake" without specifying the recipe.	Specifies the exact steps and actions to achieve the desired state. It's like giving a detailed recipe for baking a cake, including each ingredient and step.

Declarative IaC is often favored because it abstracts the complexity of implementation and allows for greater flexibility in how the infrastructure is provisioned. It's also more intuitive and aligns with the concept of treating infrastructure as code. Imperative IaC, on the other hand, provides more direct control over the process but might require more effort and can be less maintainable as infrastructure evolves.

In practice, many IaC tools support a mix of both approaches, allowing you to define high-level structures declaratively while including imperative instructions for specific actions if needed.

Summary

Conclusion

In conclusion, this module introduced you to the ground-breaking concept of Infrastructure as Code (IaC). We explored how IaC transforms infrastructure deployment by treating it as code, enhancing efficiency, consistency, and collaboration. You've gained insights into IaC's benefits, like automation and version control, while acknowledging potential complexities. Furthermore, we discussed essential IaC tools, including Terraform, Ansible, and Bicep. Understanding how IaC aligns with DevOps practices equips you with valuable insights as we progress to deeper aspects of automation and collaboration in the DevOps journey.

Next Steps

- Amazon Web Services. 2023. AWS CloudFormation. [Webpage] Retrieved from https://aws.amazon.com/cloudformation/
- Ansible. Red Hat Inc. 2023. Automation for everyone. [Webpage] Retrieved from https://www.ansible.com/
- HashiCorp. Terraform. N.d. Automate infrastructure on any cloud with terraform. [Webpage] Retrieved from https://www.terraform.io/
- Microsoft DevOps. What is infrastructure as code (IaC). [Webpage] Retrieved from https://learn.microsoft.com/en-us/devops/deliver/what-is-infrastructure-as-code
- Microsoft. 2023. What is Bicep? [Webpage] Retrieved from https://learn.microsoft.com/en-us/azure/azure-resource-manager/bicep/overview?tabs=bicep

Module 3: Version control for infrastructure

Git records changes as commits and lets people work on separate branches before combining their work. For IaC, that history explains why infrastructure changed, who reviewed it, and which configuration produced an environment.

Expanded explanation

Working tree, staging area, and commit

Your working tree contains current edits. Staging selects exactly which edits belong in the next commit. The commit is a named snapshot with a message. Small, focused commits make reviews and reversions easier than one large commit containing unrelated changes.

Local and remote repositories

A local repository lets you inspect history and commit without a network connection. A remote repository is a shared copy used for collaboration and backup. Fetching brings remote history into the local repository; pushing publishes local commits; pulling typically fetches and integrates remote changes.

Branches and reviews

A branch gives a change its own line of development. A typical workflow creates a branch, changes the IaC files, runs checks, asks for review, and merges only after the proposed infrastructure plan is understood. Branches reduce interference, but they do not replace communication about shared resources.

IaC repository hygiene

Keep reusable modules, environment inputs, and documentation in a predictable structure. Do not commit passwords, keys, or generated state containing secrets. Add generated files to ignore rules where appropriate, and use a secure state backend and secret-management process.

Understanding a change before merge

A code diff shows what was edited; an infrastructure plan shows what the tool expects to create, update, or remove. Review both. A harmless-looking text change can have a large infrastructure effect, especially when names, identifiers, or dependency relationships change.

Practical example

A teammate proposes a network rule on a feature branch. The review includes the file diff, a validation result, and the IaC plan. The reviewer notices that the rule allows traffic from every address instead of the office range. The team fixes the branch before merging, leaving the reason and correction in the project history.

Key point

For IaC, Git is both a collaboration tool and an audit trail. A safe merge requires understanding the infrastructure effect, not only the text diff.

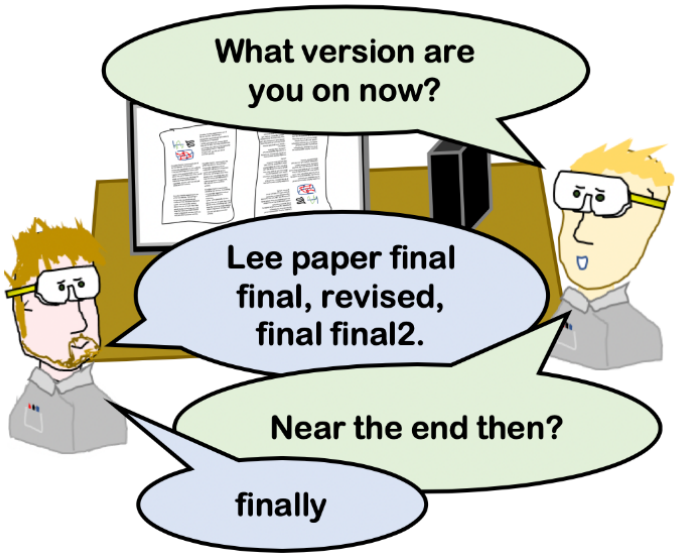
College lesson notes

Introduction

Module 3 : Version Control

Welcome to the "Version Control for Infrastructure Deployment Scripts" module! In this segment of our course, we will delve into the world of version control systems and explore how Git, one of the most powerful and widely used version control systems, can revolutionize the way you manage and deploy infrastructure scripts. Why Version Control Matters?

Imagine working on a complex project with multiple team members, each contributing code, scripts, and configurations. Something like this image. As changes accumulate:



- How can you keep track of who did what, when, and why?
- How do you manage different versions of scripts and swiftly revert to a working state when things go awry?

This is where version control comes to the rescue. Version control systems are tools designed to manage the evolution of your code and scripts. They provide a structured way to track changes, collaborate efficiently, and safeguard your work from accidental loss or errors.

Learning Objectives

By the end of this module, you should be able to complete the following:

- Explain the concept of version control and its relevance in software development and infrastructure as code (IaC)
- Interpret the primary functionalities of Git as a version control system.
- Implement basic Git commands for managing local and remote repositories
- Apply best practices in version controlling infrastructure deployment scripts

Key Terms and Concepts

Key Term or Concept	Definition
CI/CD	Continuous Integration and Continuous Delivery (or Continuous Deployment).
IaC	Infrastructure as Code

Version Control

Version control is a system that helps manage changes to files and projects over time. It allows multiple contributors to collaborate on a project efficiently, track changes, and maintain a history of modifications. Version control offers several benefits, including:

- Collaboration: Multiple team members can work on the same project simultaneously.
- History Tracking: You can review, revert, or analyze changes made to your codebase.
- Branching and Merging: Developers can work on separate features without interfering with each other.
- Backup and Recovery: Protects against data loss and provides the ability to recover earlier versions.
- Code Review: Facilitates code review processes by comparing changes.

Git

Git is a distributed version control system widely used in software development. It's lightweight, fast, and designed to handle everything from small to large projects with speed and efficiency.

Functionalities

The Primary Functionalities of Git are as follows:

- Repository: A repository is a directory where Git tracks your project. It contains all the project's files and the complete history of changes.
- Commit: A commit is a snapshot of the project at a specific point in time. Each commit has a unique identifier and includes a message describing the changes.
- Branching: Git allows you to create branches, which are independent lines of development. They are useful for isolating features or bug fixes without affecting the main codebase.
- Merging: Merging combine’s changes from one branch into another. It's commonly used to integrate completed features back into the main codebase.
- Pull and Push: Pull fetches changes from a remote repository and integrates them into the current branch. Push uploads local commits to the remote repository.
- Remote Repositories: Remote repositories are versions of your project hosted on a server, often used for collaboration. The most popular remote hosting service for Git is GitHub.

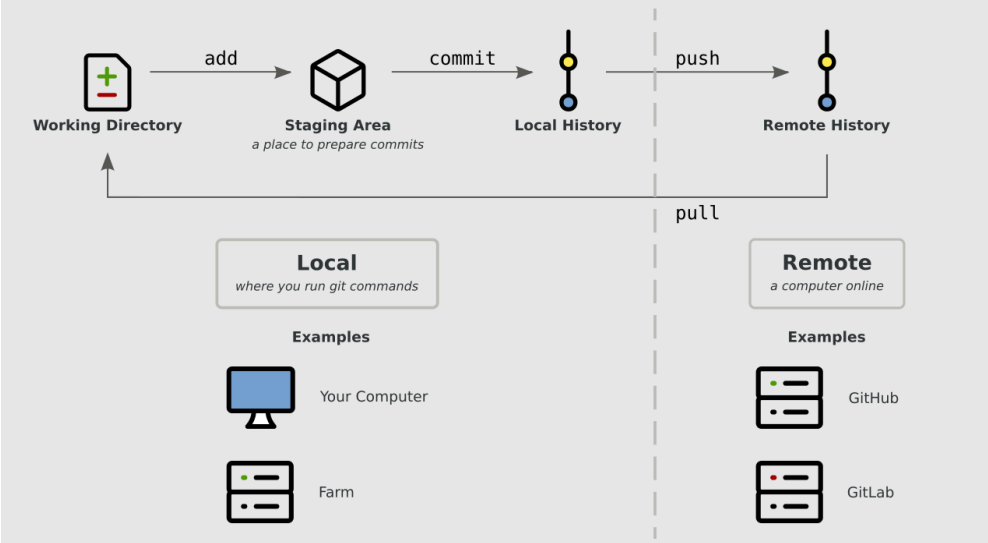
Basic Git Commands: git init : Initializes a new Git repository in the current directory. git clone [repository_url] : Copies a remote repository to your local machine. git add [file_name] : Stages a file for commit. git commit -m "commit_message" : Creates a commit with staged changes. git pull [remote] [branch] : Fetches changes from a remote repository and merges them into the current branch. git push [remote] [branch] : Uploads local commits to a remote repository. git branch : Lists all branches in the repository. git checkout [branch_name] : Switches to the specified branch. git merge [branch_name] : Merges changes from one branch into another.

Local Vs Remote Repositories

Local repositories and remote repositories are fundamental concepts in version control systems like Git. They serve different purposes in managing and collaborating on code. Let's explore the key differences between local and remote repositories.

Local Repository	Remote Repository
Definition: A local repository is a copy of the project's entire codebase that resides on your local machine. Location: Local repositories are stored on your computer's hard drive or a network drive accessible from your computer. Functionality: You can create, edit, and commit changes to files in the local repository. You can create branches, merge branches, and revert changes. Commit history and version control are managed locally. Collaboration: Local repositories are primarily for individual development and testing. Changes made in a local repository aren't automatically shared with others. Advantages: Faster access and operations compared to remote repositories. Useful for experimenting and making changes without affecting others.	Definition : A remote repository is a version of your local repository hosted on a remote server or a web-based platform. Location: Remote repositories are stored on a server, often accessed via the internet. Functionality: Remote repositories provide a centralized location for collaboration and code sharing. Changes made to a remote repository are visible to other collaborators. Collaboration: Multiple developers can collaborate by pushing and pulling changes to/from the remote repository. Remote repositories facilitate concurrent work on the same codebase. Advantages : Enables teamwork and collaboration across locations. Acts as a backup and central reference for the project's codebase. Supports code review, issue tracking, and project management.

These differences are illustrated as follows:



Branching in Git

Branching is a fundamental concept in Git that allows you to work on multiple independent lines of development within a single repository. Each branch represents a snapshot of your code base, and you can create, switch between, merge, and delete branches to manage different features, bug fixes, or experiments. Branching enables parallel development and prevents conflicts when multiple developers work on the same codebase simultaneously.

Here's a breakdown of the concept of branching in Git:

1. Default Branches:

In a Git repository, you typically start with two main branches:

- master (or main): This is the default branch representing the stable and production-ready code.
- develop: This is a branch where new features and bug fixes are integrated for testing before being merged into the master branch.

2. Creating a New Branch:

To create a new branch, you can use the git checkout -b command followed by the branch name:

git checkout -b new-feature This creates a new branch named new-feature and switches to it

3. Working on a Branch:

After creating a new branch, you can make changes to your code as needed. Any commits you make will be specific to the active branch.

4. Switching Between Branches:

You can switch between branches using the git checkout command:

git checkout another-branch

This switches you to the another-branch and updates your working directory to reflect its contents.

5. Merging Branches:

Merging allows you to combine the changes from one branch into another. To merge a branch into the current branch, you can use:

git merge branch-to-merge

This combines the changes from branch-to-merge into the current branch.

6. Resolving Merge Conflicts:

Sometimes, when merging branches, Git might encounter conflicting changes. You'll need to manually resolve these conflicts by editing the conflicting files and then committing the resolved changes.

7. Deleting Branches:

After a branch has served its purpose, you can delete it using the -d flag:

git branch -d branch-to-delete

8. Remote Branches:

Remote branches are branches stored on a remote server, such as GitHub. You can push your local branches to the remote repository to collaborate with others.

9. Pull Requests:

Pull requests (GitHub) or merge requests (GitLab) are mechanisms for proposing and discussing changes before merging them into a target branch. They often involve code review, discussions, and automated testing.

Best Practices for Version Controlling of IaC

Here, we are going to briefly mention some of the best practices for version control of your IaC.

Repository Structure: Organize your repository to separate code and deployment scripts. Use directories to distinguish different components.

Use Descriptive Commit Messages: Write clear commit messages that explain the purpose of the changes.

Use Feature Branches: Create separate branches for each feature or change to maintain a clean history.

Use .gitignore: Specify files or directories that should not be tracked by Git, like sensitive configuration files or build artifacts.

Documentation: Include README files and inline comments to explain the purpose and usage of deployment scripts.

Summary

Conclusion

Version control is your best friend when dealing with infrastructure deployment scripts. It helps manage changes, encourages teamwork, and provides a safety net in case of emergencies. By adopting practical Git practices, you'll streamline your workflow, enhance collaboration, and ensure the reliability of your infrastructure deployments.

References

Afreen, Sana. Simplilearn. 2023. All The Git Commands You Need to Know About. [Webpage] Retrieved from https://www.simplilearn.com/tutorials/git-tutorial/git-commands

Atlassian Bitbucket. Git commands. [Webapge] Retrieved from https://www.atlassian.com/git/glossaryKulkarni, Vijay Kedar. Opensource.com. 2018. A Step-by-Step Guide to Git. [Webpage] Retrieved from https://opensource.com/article/18/1/step-step-guide-git

Villalobos, Ray. LinkedInLearning. 2021. Learning Git and GitHub. Retrieved from What is Git? (linkedin.com)

Module 4: Continuous integration and delivery

A CI/CD pipeline turns a code change into a sequence of automated checks and deployment steps. It provides fast feedback to the author and a traceable route from source code to a running version.

Expanded explanation

Pipeline stages

A common sequence checks out source, builds it, runs static checks and tests, creates a versioned artifact, deploys to a test environment, performs further checks, and then releases to production. The precise order varies, but each stage should have a clear purpose and failure condition.

Artifacts and promotion

An artifact is the built output, such as a container image or package. Build once and promote the same version through environments when possible. Rebuilding for each environment can produce subtly different outputs and make a production problem harder to reproduce.

Quality gates

A gate stops a change when a required check fails. Unit tests, integration tests, security checks, approval steps, and deployment health checks are possible gates. More gates are not always better; choose checks that catch meaningful risks without making feedback unnecessarily slow.

Open-source and cloud tools

Open-source CI/CD platforms can offer broad customization and portability but may require more maintenance. Cloud vendor services usually integrate closely with that vendor's identity, hosting, and monitoring tools. Choose based on the team's environment, governance needs, operations effort, and ability to move between platforms.

Feedback after release

A successful deployment command is only one signal. The pipeline or operations workflow should also check whether the service starts, responds correctly, and stays healthy. A rollback or fix-forward plan makes failures less disruptive.

Practical example

An e-commerce team changes checkout code. A pipeline compiles the application, runs unit tests, tests the checkout flow against a temporary database, and creates a versioned image. The image goes to staging for a smoke test. A failed checkout smoke test blocks production, while the failure report points the developer to the broken behavior.

Key point

A pipeline is a decision system as well as an automation script: it records evidence that a particular version is ready for the next environment.

College lesson notes

Introduction

Module 4 : Continuous Integration and Continuous Deployment (CI/CD)

Welcome to Module 4. Here we will dive into the essential concepts of Continuous Integration and Continuous Deployment (CI/CD). In today's fast-paced software development landscape, delivering high-quality applications quickly and efficiently is paramount. CI/CD practices have emerged as indispensable methodologies that empower development teams to achieve just that.

In this module, we will embark on a journey to comprehend the key concepts, tools, and strategies that drive CI/CD. We will explore the significance of CI/CD within the larger software development lifecycle and understand how it contributes to maintaining version control, enhancing code quality, and accelerating project timelines. By the end of this module, you will have a firm grasp of both the theory and practical aspects of CI/CD, enabling you to make informed decisions and implement these practices effectively.

Learning Objectives

By the end of this module, you should be able to complete the following:

- Explain the Key Concepts of Continuous Integration and Continuous Deployment (CI/CD)
- Compare Various Open-Source CI/CD Tools
- Evaluate Cloud Vendor-Provided CI/CD Tools
- Recognize the Significance of CI/CD in the Version Control System
- Interpret the ease of Use, Customization Options, Community Support, and Cost Associated with these Tools

Key Terms and Concepts

Key Term or Concept	Definition
CI/CD	Continuous Integration and Continuous Delivery (or Continuous Deployment).
IaC	Infrastructure as Code

Defining Continuous Integration and Continuous Deployment (CI/CD)

Continuous Integration and Continuous Deployment (CI/CD) are a set of practices in software development that emphasize automating and streamlining the processes of integrating code changes, testing them thoroughly, and deploying them to production environments. The primary goal of CI/CD is to ensure the rapid and reliable delivery of high-quality software, enabling development teams to iterate quickly, respond to market demands, and maintain a consistent user experience.

Real-World Example Consider an e-commerce company that specializes in selling consumer electronics. The company's software development team is responsible for maintaining and enhancing their online shopping platform.

Here's how CI/CD practices could be applied in this real-world scenario:

1. Continuous Integration:

Whenever a developer working on the e-commerce platform completes a feature or fixes a bug, they commit their code changes to a shared version control repository, such as Git. The CI process automatically triggers a series of actions:

- The new code changes are fetched from the repository and merged with the existing codebase.
- Automated unit tests, integration tests, and possibly other types of tests are executed to ensure that the code behaves as expected and does not introduce regressions.
- Code quality checks, such as code style conformity and static code analysis, are performed.

If all tests and checks pass successfully, the code changes are considered integrated and ready for the next step.

2. Continuous Deployment:

With CI complete, the code changes move into the CD pipeline for deployment. In this scenario, the company practices Continuous Deployment, which means that once code changes pass all tests, they are automatically deployed to the production environment without manual intervention. Here's how it unfolds:

- The CI/CD pipeline deploys the new code changes to a staging or pre-production environment.
- In this environment, further testing is conducted, including user acceptance testing (UAT) and performance testing, to ensure that the changes work well in a real-world setting.
- If the tests in the staging environment pass successfully, the code is automatically deployed to the production environment, making the new feature or bug fix available to users.

Benefits in Action

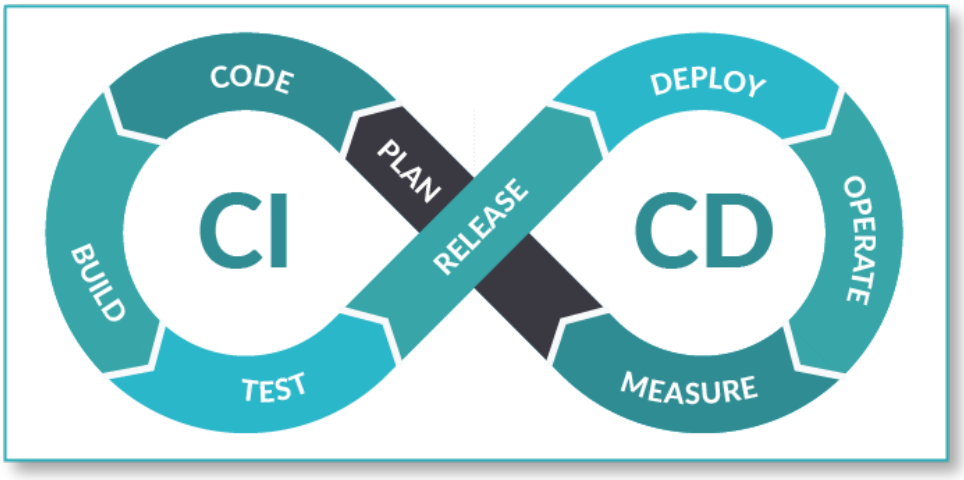
The CI/CD practices offer numerous benefits to the e-commerce company. These benefit include, but are not limited to:

- Faster Time-to-Market: Developers can work on features and bug fixes independently, knowing that the CI/CD pipeline will catch any issues early.
- Reliable Releases: Automated testing reduces the risk of bugs and ensures that only stable and well-tested code reaches production.
- Immediate Feedback: Any integration or testing failures are detected and communicated back to developers quickly, allowing them to address issues promptly.
- Agile Adaptation: The company can respond rapidly to market demands, deploying new features and improvements with minimal delay.
- Consistency: The automated deployment process guarantees that each release is consistent and follows the same steps, reducing the chance of human errors.

CI/CD Steps

The CI/CD pipeline consists of a series of steps that guide the journey of code changes from planning to deployment and operation. Each step serves a distinct purpose and contributes to the overall goal of delivering high-quality software efficiently. Here's an explanation of each step in the CI/CD pipeline:

Let’s break down the processes illustrated, a little further.



1. Plan:This is the initial phase where development tasks are planned and prioritized. It involves defining the features, bug fixes, and improvements that need to be implemented. Project managers, developers, and stakeholders collaborate to outline the scope and requirements for the upcoming work.

2. Code:During this phase, developers write and modify code to implement the planned features or fixes. The code is developed according to coding standards and best practices, and it's crucial to maintain version control to keep track of changes and facilitate collaboration.

3. Build:After writing the code, it needs to be compiled or transformed into an executable form. The build process involves converting source code files into machine-readable binaries or artifacts. This phase ensures that the code can be executed and tested.

4. Test:Testing is a critical part of the CI/CD pipeline. It involves multiple types of tests, including unit tests, integration tests, and end-to-end tests. These tests verify the functionality, performance, and reliability of the code. Automated testing ensures that the code behaves as expected and does not introduce regressions.

5. Release:Once the code passes the tests, it's considered for release. The release phase involves preparing the code, artifacts, and any required configuration for deployment. It ensures that the release is well-packaged and ready to be deployed.

6. Deploy:Deployment involves taking the prepared code and releasing it into various environments, such as staging, testing, and production. Automated deployment processes ensure consistency and reduce the likelihood of errors during the deployment process.

7. Operate:After deployment, the software is live in the production environment. The operation phase involves monitoring the application's performance, availability, and any potential issues. Continuous monitoring ensures that the software remains stable and responsive to user interactions.

8. Measure:Measurement and monitoring play a significant role in the CI/CD pipeline. Key performance indicators (KPIs) are tracked to assess the software's health, user experience, and business impact. Data collected during the operation phase provides insights into how well the software is performing and where improvements can be made.

Open-Source CI/CD Tools

Open-source tools such as Jenkins, Travis CI, and Circle CI are widely used due to their cost-effectiveness, flexibility, and robust community support. Their open-source nature allows teams to adapt and customize the tools to their unique workflow requirements, driving innovation. An active community contributes to the tool's development, providing quick fixes, updates, and a wealth of shared knowledge and resources. For the purpose of this module, let’s take a look at Jenkins, Travis CI and Circle CI in a little more detail.

1. Jenkins:Jenkins is one of the most widely used open-source CI/CD tools. It provides a flexible and extensible platform for automating various aspects of the software development lifecycle. Some of its features include:

Extensibility

Jenkins supports a wide range of plugins that allow you to integrate with different tools and technologies.

Customization

Its open-source nature allows you to tailor the tool to fit your specific requirements.

Workflow Support

Jenkins supports defining complex workflows as code, enabling teams to create sophisticated build and deployment pipelines.

Community and Documentation

Jenkins has a large and active community, offering extensive documentation and resources for users at all skill levels.

Common Problem

Plugin compatibility issues, users may occasionally face issues due to plugin incompatibility.

---

2. Travis CI:Travis CI is a popular open-source CI/CD platform that is especially favored by projects hosted on GitHub. Some of its features include:

GitHub Integration

Travis CI seamlessly integrates with GitHub repositories, making it easy to set up and manage builds for open-source projects.

Configuration as Code

Build configurations are defined in a .travis.yml file in the repository, allowing for version-controlled, reproducible builds.

Parallel Testing

Travis CI supports running tests in parallel, speeding up build times for projects with extensive test suites.

Easy Setup

Its user-friendly setup process makes it accessible to developers, regardless of their CI/CD experience.

Common Problem

Limited customization compared to Jenkins.

---

3. CircleCI:

CircleCI is a versatile CI/CD platform that offers both cloud-based and self-hosted solutions. Some of its features include:

Orbs

CircleCI's reusable configuration elements, called "Orbs," simplify the setup and management of complex build pipelines.

Matrix Builds

CircleCI supports matrix builds, allowing you to test your code against multiple environments and configurations in parallel.

Workflows

Teams can define sophisticated workflows that encompass various stages of development, testing, and deployment.

Scalability

With cloud-based options, CircleCI can scale to handle large projects and varying workloads.

Common Problem

No free private projects, it does not offer free services for private projects, unlike Jenkins and Travis CI.

These open-source CI/CD tools have indeed gained popularity due to their affordability, flexibility, and vibrant communities. They empower development teams to automate repetitive tasks, reduce errors, and accelerate the delivery of software. As the open-source community continues to contribute to these tools, they evolve with the changing needs of modern software development, ensuring that teams have access to the latest features and improvements.

Cloud Vendor-Provided CI/CD Tools

Cloud vendors offer a range of CI/CD tools that integrate seamlessly with their respective cloud platforms. Let's explore three prominent cloud vendor-provided CI/CD tools:

- AWS CodePipeline from Amazon Web Services
- Azure Pipelines from Microsoft Azure; and
- Google Cloud Build from Google Cloud Platform.

1. AWS CodePipeline (Amazon Web Services):AWS CodePipeline is a fully managed CI/CD service that automates the building, testing, and deployment of code changes. Here's an overview of its features.

Integration

LTight integration with other AWS services like CodeCommit, CodeBuild, and CodeDeploy.

Visual Pipeline Designer

Provides an intuitive visual interface for creating and managing complex CI/CD pipelines.

Artifact Management

Supports the management and storage of build artifacts.

Third-Party Integrations

Extensible with third-party tools and services via AWS Lambda.

Security

Integrates with AWS Identity and Access Management (IAM) for granular access control.

---

2. Azure Pipelines (Microsoft Azure):Azure Pipelines is a CI/CD platform provided by Microsoft Azure. It offers flexibility and robust integration with Azure services and other platforms.

YAML Configuration

Pipelines are defined using YAML, making it easy to version control and reproduce.

Build and Release Pipelines

Supports both build pipelines (for compiling code) and release pipelines (for deployment).

Parallelism

Enables parallel execution of jobs and stages for faster builds.

Multi-Platform

Supports building and deploying applications on various platforms including Windows, Linux, and macOS.

Integration

Seamless integration with Azure services, GitHub, Bitbucket, and others.

---

3. Google Cloud Build (Google Cloud Platform):Google Cloud Build is a CI/CD platform that focuses on building, testing, and deploying applications on Google Cloud Platform.

Docker Support

Builds can be containerized using Docker and pushed to Google Container Registry.

Configuration as Code

Pipelines are defined using a YAML-based configuration.

Triggers

Automated triggers can initiate builds when code is pushed to source repositories.

Integration

Integrates with Google Cloud services like Google App Engine, Google Kubernetes Engine, and others.

Parallel Builds

Supports parallel execution of build tasks.

Considerations for Choosing:When evaluating these cloud vendor-provided tools, consider factors such as integration with your existing cloud platform, ease of use, flexibility, scalability, and cost. These tools often work seamlessly with other services within their respective cloud ecosystems, providing a cohesive experience for developers.

OverviewCloud vendor-provided CI/CD tools can be beneficial if you are already invested in a particular cloud platform. They offer streamlined integration, allowing you to leverage other cloud services alongside your CI/CD processes. Choose the tool that aligns with your project's requirements, team's familiarity, and long-term development strategy.

Open-source Vs cloud vendor-provided CI/CD tools

Knowing what we know about Open Source CI/CD Tools and Cloud Vendor-provided CI/CD Tools, let’s compare the attributes of both.

Attribute	Open-Source CI/CD Tools	Cloud Vendor-Provided CI/CD Tools
Customization Options	High	Limited (within the platform's framework)
Community Support	Strong, active community	Community support varies, but often available
Flexibility in Toolchains	High	May be limited to vendor-supported integrations
Integration with Cloud Services	Limited or requires custom configuration	Seamless integration with cloud services
Scalability	Depends on self-hosted infrastructure	Built-in scalability with cloud resources
Reliability	Depends on self-maintenance and configuration	Reliable due to integration with cloud services
Maintenance Overhead	May require more maintenance and updates	Reduced maintenance due to managed services
Cost	Often lower upfront cost, but consider hosting, maintenance, and support costs	Subscription-based pricing with variable tiers, consider usage and features offered

Summary

Conclusion

As this module comes to an end, you have now examined the CI/CD pipeline stages, benefits of automation, and compared open-source tools (e.g., Jenkins) with cloud vendor tools (e.g., AWS CodePipeline). You learned that Open-source tools offer customization and community support, while vendor tools provide integration, scalability, and reduced maintenance. This knowledge equips you to choose the right tools for efficient software delivery and maintenance.

References

Circle Internet Services, Inc. 2023. Getting Started with CircleCI. Quick Start Guide [Webpage] Retrieved from https://circleci.com/docs/getting-started/

Google Cloud Documentation. N.d. CI/CD on Google Cloud Platform [Webpage] Retrieved from https://cloud.google.com/docs/ci-cd

LearnTek. DEVOPS. 2017. Why Jenkins is a Great Tool for DevOps [Webpage] Retrieved from https://www.learntek.org/blog/jenkins-great-tool-devops/Microsoft. 2023. Azure DevOps Pipelines [Webpage] Retrieved from https://azure.microsoft.com/en-ca/products/devops/pipelines

---

Module 5: Terraform: defining infrastructure

Terraform lets a team describe resources in HashiCorp Configuration Language (HCL). Providers connect Terraform to cloud and service APIs, while resources express the objects that should exist and the relationships among them.

Expanded explanation

Configuration and resource graph

A Terraform configuration contains blocks such as providers, resources, variables, data sources, and outputs. References between blocks create dependencies. Terraform uses those dependencies to decide a safe order for creating or changing resources.

The main workflow

Initialization prepares providers and modules. Validation checks configuration structure. Planning compares the desired configuration with known state and shows proposed actions. Applying performs the approved plan. Review a plan before apply, particularly when it proposes replacement or deletion.

Variables and outputs

Variables make a configuration reusable across environments without duplicating every file. Outputs expose selected results, such as a resource identifier or address. Inputs should be validated and sensitive values handled through an appropriate secret process rather than copied into source control.

State and lifecycle

Terraform state maps configuration addresses to managed real-world objects. Protect it because it may contain sensitive information and because concurrent writes can corrupt the management workflow. Some changes can be updated in place; others require replacement when a property cannot be changed on an existing resource.

Modules and maintainability

A module groups reusable resources behind inputs and outputs. Modules reduce duplication, but a module should have a clear responsibility and sensible defaults. Reuse is helpful when it makes environments more consistent, not when it hides important choices.

Illustrative example

```
variable "location" {
  type = string
}

resource "azurerm_resource_group" "app" {
  name     = "rg-example-app"
  location = var.location
}

output "resource_group_id" {
  value = azurerm_resource_group.app.id
}
```

Practical example

This illustrative HCL defines one Azure resource group, takes its location as an input, and exposes its ID as an output. A full working configuration also needs an appropriate provider block, authentication, and a reviewed plan. The reference in the output points to the managed resource rather than repeating its name.

Key point

Terraform is most predictable when configuration, state, variables, and the proposed plan are all treated as part of one controlled change.

College lesson notes

Introduction

Introduction to Terraform - Part I

In this module, we will dive into the world of infrastructure as code (IaC) and explore the powerful tool known as Terraform. As modern technology environments demand agility, scalability, and efficiency, Terraform has emerged as a go-to solution for managing and provisioning infrastructure resources. By the end of this module, you will have a solid understanding of what Terraform is, how to create scripts with it, and its relevance to Continuous Integration and Continuous Deployment (CI/CD) processes.

Learning Objectives

By the end of this module, you should be able to complete the following:

- Recognize the role of Terraform in Infrastructure as Code (IaC): The students will gain an in-depth understanding of Terraform, its role in IaC, and its importance in automating infrastructure provisioning.
- Create, manage, and modify Terraform scripts to meet different deployment needs.
- Test infrastructure scripts using Terraform, ensuring they are error-free and functional.

Key Terms and Concepts

Key Term or Concept	Definition
CI/CD	Continuous Integration and Continuous Delivery (or Continuous Deployment).
IaC	Infrastructure as Code

Terraform

Terraform is an open-source infrastructure as code tool developed by HashiCorp. It enables you to define, manage, and provision infrastructure resources using declarative configuration files. With Terraform, you can describe your desired infrastructure state and version it just like you would with application code. This approach ensures consistent and repeatable infrastructure deployments across various environments.

Key Concepts

Infrastructure as Code (IaC): The practice of defining and managing infrastructure resources using code and automation.

Declarative Syntax: Terraform uses a declarative language to specify what you want your infrastructure to look like, rather than scripting how to achieve that state.

Resources: In Terraform, resources represent the building blocks of your infrastructure, such as virtual machines, networks, databases, and more.

Providers: Terraform supports various cloud providers (e.g., AWS, Azure, Google Cloud) and on-premises infrastructure. Providers allow you to interact with these platforms through Terraform.

Benefits in Action

Using Terraform has several advantages over manually managing your infrastructure. Some of these advantages include, but are not limited to:

Let’s expand on the advantages further.

1. Orchestrates Infrastructure, not Just Configuration:Terraform goes beyond traditional configuration management tools by orchestrating the entire lifecycle of infrastructure resources. It not only provisions resources but also manages their creation, updates, and deletion. This allows you to define complex relationships and dependencies between resources, ensuring they are provisioned in the correct order and maintained consistently.

2. Multi-Provider Support:Terraform provides a unified approach to managing infrastructure across a variety of cloud providers and platforms. This multi-provider support means you can define infrastructure in a consistent way regardless of whether you're using AWS, Azure, Google Cloud, or any other supported provider. This flexibility is invaluable when dealing with hybrid or multi-cloud environments.

3. Immutable Infrastructure:One of Terraform's key principles is the concept of immutable infrastructure. This means that instead of modifying existing resources, you recreate them from scratch when changes are needed. This approach ensures that your infrastructure remains consistent and predictable, minimizing configuration drift and reducing the risk of unexpected issues caused by incremental changes.

4. HashiCorp Configuration Language (HCL):Terraform uses HCL, a declarative and easy-to-understand configuration language. HCL is designed to be human-readable and is specifically tailored for defining infrastructure as code. Its syntax resembles JSON but is more expressive and allows you to define resources and relationships with concise, clear statements.

5. Infrastructure as Code (IaC) Paradigm:Terraform aligns perfectly with the IaC paradigm, enabling you to manage infrastructure using the same practices as software development. This means versioning your infrastructure configurations, collaborating with team members through version control systems, and automating deployment and testing processes.

6. Easily Portable to Any Provider:One of the most powerful features of Terraform is its portability. Once you've written Terraform configurations for one provider, they can often be easily adapted to work with another provider. While there may be some differences due to specific provider features, the core infrastructure definitions remain largely consistent. This saves time and effort when switching or expanding your infrastructure environment.

7. Infrastructure as a Collaborative Process:Terraform promotes collaboration among teams. By using version control systems like Git, teams can collaboratively develop, review, and test infrastructure code. This shared approach enhances transparency, helps catch errors early, and ensures that everyone is working with the same version of the infrastructure.

8. Predictable and Reproducible Deployments:With Terraform, deployments are consistent and repeatable. Infrastructure changes are executed according to the defined configuration, reducing the chances of human error and enabling consistent outcomes across environments. This predictability is crucial for maintaining the stability and reliability of your systems.

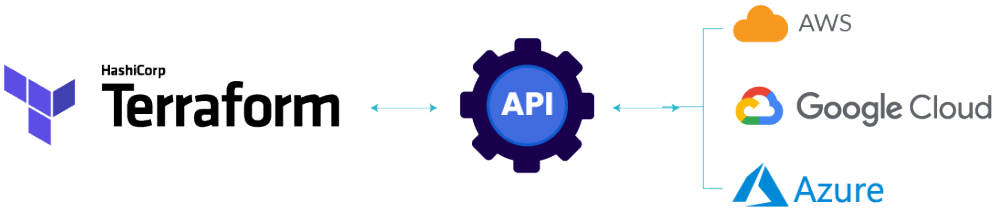
9. Infrastructure Evolution and Rollbacks:Terraform enables you to track changes over time, making it easy to evolve your infrastructure configurations while maintaining a historical record. If a change introduces unexpected issues, Terraform's versioning and state management capabilities make it possible to rollback to a previous known good state.

10. Third-Party Ecosystem and Modules:Terraform boasts a vibrant ecosystem of community-contributed modules, enabling you to reuse best practices, common patterns, and pre-built infrastructure configurations. This accelerates development and ensures that you're following recommended approaches for various scenarios.

How does Terraform work?

Terraform functions by leveraging APIs, which serve as interfaces to interact with cloud platforms and various services. In simpler terms, terraform is capable of managing resources on diverse cloud providers and services that offer accessible APIs. This means that platforms such as Azure, Google Cloud, or AWS, which provide accessible APIs, can be effectively orchestrated and managed using Terraform.

Terraform works by leveraging APIs to manage resources on various cloud platforms and services.



Writing Terraform Code

Understanding the HashiCorp Configuration Language (HCL)

Terraform is a tool that allows you to define and manage your infrastructure as code. This means you can describe your desired infrastructure configuration using text-based files rather than manually configuring resources through a graphical user interface.

Two common languages can be used to write these configuration files:

- JSON; and
- HashiCorp Configuration Language (HCL).

HCL Is More Commonly Used Due to Its Readability and Simplicity:HCL's design philosophy centers on making configuration files highly readable, writable, and easy to understand. This emphasis on readability leads to more maintainable code and better collaboration within teams.

Basic HCL Syntax:The syntax of HCL is designed to be easy to read and write. Here's a simple breakdown:

- Configuration blocks are enclosed in curly braces {}.
- Each block has a block type (e.g., resource, variable, data) and may have a label.
- Within the block, you define arguments using the KEY = VALUE syntax.

Example: Let's define an AWS S3 bucket resource using HCL:

In this example: resource is the block type. aws_s3_bucket is the resource type. "my_bucket" is the label used to reference this resource in other parts of the configuration. bucket and acl are the arguments with their respective values.

main.tf > ...

```
# Define an AWS S3 bucket resource
resource "aws_s3_bucket" "my_bucket" {
  bucket = "my-unique-bucket-name"
  acl    = "private"
}
```

Variables in HCLVariables allow you to parameterize your configuration and make it more flexible. You can define variables in separate .tf files or inline within your configuration.

In this example, the variable "region" is defined in a separate variables.tf file. It has a description and a default value. In the main.tf file, the variable is used to set the AWS region in the provider "aws" block.

```
# variables.tf

variable "region" {
  description = "AWS region for resources"
  default     = "us-west-1"
}

# main.tf

# Use the defined variable
provider "aws" {
  region = var.region
}

```

Comments in HCLComments in HCL use the # symbol

```
main.tf
1  # This is a single-line comment
2
3  /*
4     This is a multi-line
5     comment.
6  */
7
```

Terraform Lifecycle

Illustrated here is the lifecycle of Terraform.

Now, let's expand on this lifecycle.



- Terraform Init:
- Initializes the Terraform environment, setting up the necessary resources and configuration.
- Typically executed once per session or when a new project is started.
- Downloads provider plugins and prepares the working directory for configuration.
- Terraform Plan:
- Compares the desired state defined in your configuration with the current state of resources.
- Generates an execution plan that details the actions Terraform will take to achieve the desired state.
- Does not make any actual changes to the deployed resources; it's a preview of changes.
- Terraform Apply:
- Executes the execution plan generated during the planning phase.
- Makes changes to the actual infrastructure based on the plan.
- Resources are created, updated, or deleted to align with the desired state.
- Terraform Destroy:
- Deletes all resources managed by the specific Terraform environment.
- Reverts the infrastructure to its initial state before Terraform was applied.
- Useful for decommissioning or cleaning up resources.

In a nutshell, the Terraform lifecycle progresses from initialization and planning to applying changes and, if needed, destroying resources. This systematic approach ensures controlled and predictable management of your infrastructure throughout its lifecycle.

Terraform Configuration Files

The .tf extension is used for Terraform configuration files. These files contain the HashiCorp Configuration Language (HCL) code that defines your infrastructure as code, including resources, variables, and other configuration aspects. Let's dive into more details about the .tf extension and its significance in Terraform.

Terraform Configuration Files and .tf Extension:

1. File Format

Terraform configuration files have a .tf extension. For example, main.tf, variables.tf, network.tf are all common filenames.

2. Resource Definitions

In .tf files, you define resources, variables, data sources, outputs, and other elements required to manage your infrastructure. Each resource and element follow HCL's block-based syntax.

3. Modularization

You can break down your configuration into multiple .tf files to improve organization and reusability. For instance, you might have a separate file for network resources, another for compute resources, and so on.

4. Collaboration

Storing your Terraform configuration in .tf files facilitates collaboration using version control systems like Git. Changes to your infrastructure can be tracked, reviewed, and managed just like code changes.

Example: Here's an example of a simple .tf file that defines an AWS S3 bucket resource.

main.tf > ...

```
# main.tf

provider "aws" {
  region = "us-west-1"
}

resource "aws_s3_bucket" "my_bucket" {
  bucket = "my-unique-bucket-name"
  acl    = "private"
}
```

In this example, the main.tf file contains the configuration to create an AWS S3 bucket using the aws_s3_bucket resource type.

The .tf extension is an important identifier for Terraform to recognize configuration files and understand their content. Using this extension and following HCL syntax allows you to define, manage, and evolve your infrastructure as code efficiently and systematically.

Summary

Conclusion

In this module, we explored Terraform as a potent infrastructure provisioning and management tool. We discovered how it utilizes HashiCorp Configuration Language (HCL) for defining infrastructure as code, enabling a declarative approach to specifying and achieving desired infrastructure states.

References

Daou, Karim. EXEO. 2023. Terraform - Infrastructure as Code (IAC) Benefits. [Webpage] Retrieved from https://exeo.net/en/terraform-infrastructure-as-code-iac-benefits/

Env0. 2023. Terraform Tutorial: Getting Started with Terraform. [Webpage] Retrieved from https://www.env0.com/blog/terraform-tutorial

HashiCorp. N.d Change infrastructure. [Webpage] Retrieved from https://developer.hashicorp.com/terraform/tutorials/azure-get-started/azure-change

Illarionov, Alexandra. LinkedIn Learning. 2022. Introduction to Terraform on Azure [Tutorial]. Retrieved from https://www.linkedin.com/learning/introduction-to-terraform-on-azure/terraform-state?autoSkip=true&resume=false&u=2199673

Singh, Sumit. K21Academy. 2023. Terraform Beginner's Guide: Everything you Should Know. [Webpage]. Retrieved from https://k21academy.com/terraform-iac/terraform-beginners-guide/

Module 6: Terraform: testing infrastructure code

Infrastructure code can be syntactically valid and still create the wrong environment. Testing therefore needs several layers: inexpensive checks before deployment and behavior checks against real or temporary infrastructure when the risk justifies them.

Expanded explanation

Fast checks first

Formatting and validation catch malformed or inconsistent configuration early. Static analysis and policy checks can flag patterns such as overly open network rules or missing required tags. These checks are fast, but they cannot prove that a deployed service works.

Plan review as a test of intent

A plan shows proposed creates, updates, replacements, and destroys. Compare it with the intended change. A surprise replacement is often more important than a passing syntax check because it may interrupt service or lose data.

Integration and end-to-end tests

An integration test provisions a small set of resources and verifies their interactions. An end-to-end test exercises a larger path, such as deploying an application and sending a request through it. These tests provide stronger evidence but take longer and can cost money, so temporary resources need reliable cleanup.

Terratest

Terratest is a Go testing framework often used to provision infrastructure, inspect outputs or resource behavior, assert expectations, and destroy the test environment. The test should clean up even when an assertion fails. Unique names and isolated environments reduce collisions between parallel runs.

What a useful assertion checks

A test should verify a requirement, not merely that the command returned successfully. For example, check that a load balancer endpoint responds, that only intended ports are open, or that an output value identifies the expected resource.

Practical example

Imagine a module that creates a web server. First run formatting, validation, and a plan. In a temporary test environment, apply the module and request its endpoint. Assert that the response is healthy and that the exposed port matches the design. Destroy the test resources in a guaranteed cleanup step, then review the results before promoting the module.

Key point

Testing IaC means checking both the proposed change and the behavior of the resulting environment. Each layer answers a different question.

College lesson notes

Introduction

Introduction to Terraform - II

In our part II of Terraform, we dive a little deeper into this powerful tool to explore how Terraform fits into DevOps as well as how to make digital infrastructure (like servers) using code. We'll focus on automated testing and how to watch over this infrastructure. By the end, you'll know why Terraform is useful, how to test your setups automatically, and how to keep an eye on your Terraform-made systems. This helps ensure everything runs smoothly and securely.

Learning Objectives

By the end of this module, you should be able to complete the following:

- Outline Terraform testing strategies
- Implement Testing Terraform Configuration
- Recognize the importance of testing for Infrastructure as Code (IaC) with Terraform.
- Execute automated tests for Terraform configurations using Terratest

Key Terms and Concepts

Key Term or Concept	Definition
CI/CD	Continuous Integration and Continuous Delivery (or Continuous Deployment).
IaC	Infrastructure as Code

Why Test?

Think of code like a recipe for a dish. Just as you want to make sure your recipe will result in delicious food; you want to be sure your code will work properly. Testing is like tasting the food before serving it to make sure it's tasty and safe to eat.

Think of IaC like building with LEGO blocks. Just as you'd want to ensure your LEGO creation holds together and functions as intended, testing IaC ensures your digital infrastructure is dependable and functions correctly.Furthermore, testing is important as it helps with the following:

- Preventing Future Disruptions

Testing helps identify issues before they lead to major problems. It's like checking your bicycle's tires before a long ride to prevent unexpected punctures.

Example: If you're using IaC to deploy a web application, testing can uncover any misconfigurations that might cause the app to crash or become unresponsive.

- Collaborative IaC Development

In collaborative projects, testing ensures that everyone's code for creating infrastructure components works well together. It's like making sure all the pieces of a puzzle fit seamlessly to create the full picture.

Example: Suppose multiple teams are working on different parts of an application's infrastructure. Testing helps ensure that when one team's IaC changes, it doesn't disrupt the setup of another team's components.

- Long-Term Stability:

Your IaC might work perfectly now, but will it still work a year from now? Testing guarantees that your infrastructure remains stable over time, even as technology evolves. It's like having a car that runs reliably for years without breaking down.

Example: Imagine you're using IaC to manage networking configurations. Testing ensures that even as your organization's network requirements change, your IaC can still set up and manage the connections accurately.

In summary, testing is the foundation for solid and dependable Infrastructure as Code. Just as you wouldn't build a physical structure without ensuring its stability, you shouldn't create digital infrastructure without thoroughly testing its reliability and functionality.

Terraform testing strategies

As with any code, testing strategies should be implemented in order to help validate your code and ensure that your changes dot not cause any unexpected issues or break existing functionality. Here are some of those strategies:

- Drift Testing

Drift testing is an important aspect of maintaining Infrastructure as Code (IaC) with Terraform. It involves checking for any differences between the expected state of your infrastructure, as defined in your Terraform code, and the actual state of your deployed infrastructure. The goal is to identify any configuration drift that may have occurred due to manual changes made outside of Terraform.

Terraform's Native Drift TestingTerraform provides built-in support for drift testing. When you run a terraform plan, Terraform performs a comparison between the desired state (defined in your Terraform code) and the current state (captured in the Terraform state file). It identifies any discrepancies between the two states, such as changes made directly in the cloud provider's console or CLI.

If there are differences, Terraform generates an execution plan that outlines the changes required to bring the actual infrastructure in line with the desired configuration. This execution plan indicates what resources need to be created, updated, or deleted to achieve the desired state. This gives you the opportunity to review and approve the changes before applying them with terraform apply.

Usage ExampleLet's say you've defined a virtual machine (VM) in your Terraform code, and this VM's configuration is stored in the Terraform state. If someone manually modifies the VM's attributes directly in the cloud provider's console, Terraform's drift testing will identify the differences between the actual VM configuration and what's defined in your code.

- Unit Testing

Unit testing checks individual parts of your Terraform code. You can use terraform validate to check for correct syntax and terraform plan to simulate a deployment and find potential changes.

Example: Running terraform plan will tell you if you accidentally used a wrong attribute in your code before it's deployed.

- Linting

Linters analyze your code for errors, bugs, and security issues. Tools like tflint help catch potential problems and inconsistencies.

Example: Linting might catch issues like hard-coded sensitive information in your Terraform code.

- Integration Testing

Integration testing ensures that different parts of your Terraform code work together correctly. Tools like Terratest can help test complex scenarios involving multiple modules.

Example: If your Terraform code includes both networking and virtual machine modules, integration testing can verify that they function well together.

- Compliance Testing

Compliance testing ensures your Terraform templates adhere to corporate standards and regulations. Tools like Terraform-compliance can validate if your infrastructure meets compliance requirements.Terraform-compliance is a powerful tool that allows you to define compliance rules as YAML files and test your Terraform code against those rules. This helps ensure that your infrastructure configurations adhere to security, compliance, and best practices.

Example: Compliance testing can check if your infrastructure deployment follows regional regulations and security standards.

The image below demonstrates an example of a compliance rule in YAML that specifies that an Azure Storage Account should not be publicly accessible.

```yaml
controls:
  - name: azure-storage-not-publicly-accessible
    description: Azure Storage Account should not be publicly accessible
    rules:
      - azure_storage_account:
          public_access_allowed: False
```

- End-to-End Testing

End-to-end testing combines various testing methods to test the complete deployment process, including integration between different modules.

Example: This type of testing validates the whole infrastructure setup, from networking to application deployment, to ensure all pieces work harmoniously.

- Unit Testing with CDKTF

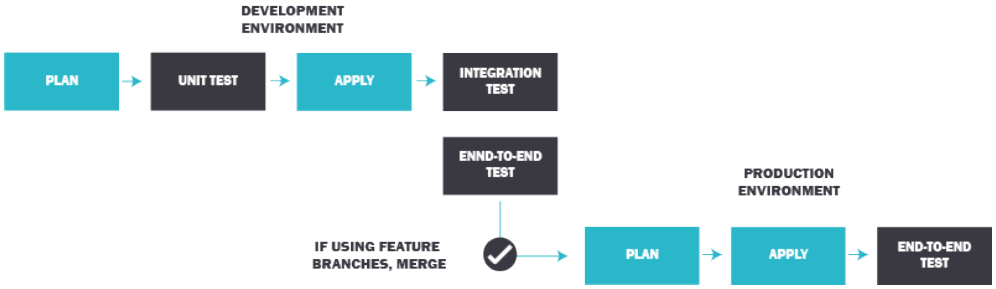
Terraform's Cloud Development Kit for Terraform (CDKTF) allows you to use familiar programming languages for infrastructure code. Unit testing in CDKTF might involve testing your infrastructure code using languages like Python, Go, or Java.

Example: With CDKTF, you can create unit tests for your infrastructure code using Python's testing frameworks.

In essence, these testing strategies help ensure that your Terraform code is accurate, functional, compliant, and works well with other components. Each strategy tackles different aspects of the development and deployment process, providing layers of confidence in your infrastructure as code.

Testing Terraform Configuration

Thorough testing of Terraform configurations is crucial to ensure the reliability and stability of your infrastructure deployments.Key points to consider when testing your Terraform configurations are as follows:



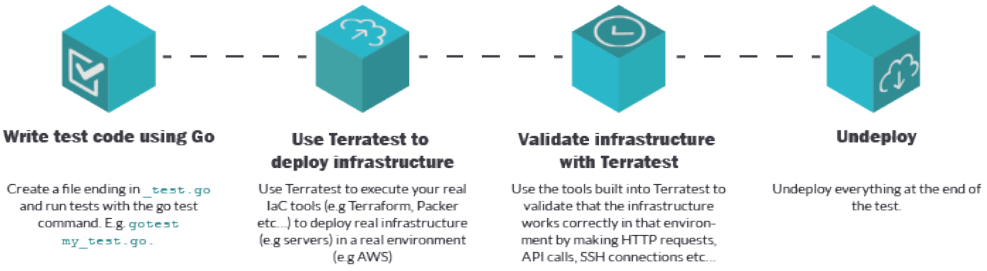
- Testing Levels: When testing your Terraform configurations, consider different levels of testing such as unit testing, integration testing, and end-to-end testing.
- Unit Testing: Focus on testing individual configuration elements that are not part of modules. Unit tests help catch syntax errors, configuration mistakes, and ensure the correctness of your code.
- Integration Testing: Integration tests ensure that changes to your configurations can be successfully applied in a development environment that closely resembles production. These tests can identify issues that may arise due to the interaction of different components.
- End-to-End Testing: End-to-end tests validate the functionality of your infrastructure in a production-like environment. They verify that your infrastructure resources work together as expected and that the initial functionality of the environment is intact.
- Development Environment: Ideally, maintain a long-running development environment that closely replicates production. This allows you to catch potential problems early and ensure that changes will not negatively impact critical business systems.
- Cost Considerations: While a fully replicated development environment might not always be feasible due to cost and complexity, create a scaled-down version of production that mimics user traffic patterns as closely as possible.
- Pipeline Integration: Incorporate Terraform testing into your continuous integration and continuous deployment (CI/CD) pipelines. This ensures that changes are thoroughly tested before being deployed to production environments.
- Flagging Errors: Be aware of the differences between development and production environments and identify which tests are critical for flagging errors. Some tests might be more disruptive to run in production, so prioritize accordingly.
- Practice Rollbacks: Testing allows you to practice applying and rolling back changes. This helps build confidence in your ability to manage and recover from unexpected issues.

By following these principles and incorporating testing into your Terraform workflow, you can identify and address potential issues early in the development process, reduce the risk of outages in production, and ensure the reliability of your infrastructure deployments.

Terratest

Terratest is a free testing framework designed to help you test your Terraform code as well as other technologies like Kubernetes, Docker, and Packer. It's a tool that automates the process of testing your infrastructure code in a controlled environment to make sure it behaves correctly and meets your requirements.

Test infrastructure code with Terratest in 4 Steps



What's unique about Terratest is that you write these tests using the Go programming language. These tests act like little programs that interact with your infrastructure to ensure everything is working as expected. Terratest comes with a bunch of tools and features that make these tests comprehensive and efficient.

With Terratest, you can set up a kind of "playground" where your code can be tested without affecting your actual systems. This helps you make sure your code works before you deploy it for real. Terratest takes care of creating the necessary resources, running tests against them, and then cleaning up after itself.

Terratest isn't just for Terraform - it can help with other technologies too, making it a versatile tool in your testing toolkit. It also supports different testing frameworks that can help you structure and manage your tests more effectively.

Using Terratest

Now that we know the key elements of Terratest, let's examine how to use it. the following setup is for testing a simple Terraform example that outputs "Hello, World!". The Terratest code tests whether the Terraform module produces the expected output.

Example

1. main.tf: This Terraform configuration defines a simple module that outputs "Hello, World!". 2. terraform_hello_world_example_test.go: This is the Terratest test file that tests the main.tf module. In this Terratest file: The TestTerraformHelloWorldExample function is the test function that runs the test. terraform.WithDefaultRetryableErrors sets up the Terraform options with retryable errors, which helps improve test stability. terraform.InitAndApply initializes and applies the Terraform configuration. terraform.Output retrieves the value of the hello_world output from the Terraform state. assert.Equal checks if the output matches the expected value "Hello, World!". To run the test, navigate to the directory where the terraform_hello_world_example_test.go file is located and use the go test command: go test -v





```
package test

import (
    "testing"

    "github.com/gruntwork-io/terratest/modules/terraform"
    "github.com/stretchr/testify/assert"
)

func TestTerraformHelloWorldExample(t *testing.T) {
    // retryable errors in terraform testing.
    terraformOptions := terraform.WithDefaultRetryableErrors(t, &terraform.Options{
        TerraformDir: "../examples/terraform-hello-world-example",
    })

    defer terraform.Destroy(t, terraformOptions)

    terraform.InitAndApply(t, terraformOptions)

    output := terraform.Output(t, terraformOptions, "hello_world")
    assert.Equal(t, "Hello, World!", output)
}
```

This will execute the Terratest test, which will run the Terraform module and verify if the output matches the expected value.

For additional information on automated tests for your infrastructure code, please familiarize yourself with the following resource. It provides practical experience for testing your infrastructure.

NOTE: Be sure to click Let's Get Started on the page.

https://terratest.gruntwork.io/

Furthermore, this next tutorial is an excellent opportunity to experience some practical elements of Terraform Testing. Familiarize yourself with the tutorial which will provide a complete step-by-step process and explanation.

Terraform testing

Summary

Conclusion

Module 6 delved into Terraform's pivotal role in DevOps and Infrastructure as Code (IaC), focusing on testing strategies. It covered the significance of testing, including unit, integration, and end-to-end tests. The module introduced tools like Terratest, which automates testing in a sandbox environment. Compliance testing and drift testing were discussed to ensure alignment with regulations and detect infrastructure changes. The module emphasized the importance of a well-structured testing pipeline to catch potential issues before deployment in a controlled and reliable manner.

References

Daou, Karim. Exeo. 2022. Terraform Infrastructure as Code (IaC) Benefits. [Webpage] Retrieved from https://exeo.net/en/terraform-infrastructure-as-code-iac-benefits/

Gruntwork.io. Terratest. 2023. Terraform Test Automation and Beyond [Webpage] Retrieved from https://terratest.gruntwork.io/

Our Cloud School. 2022. Terraform Testing | Test Azure Terraform with terratest | Terratest | Terragrunt. [YouTube] Video. Retrieved from https://www.youtube.com/watch?v=UKitApmIHFM&t=30s

Roper, Jack. Spacelift. 2023. How to test Terraform code - Strategies and Tools. [Webpage] Retrieved from https://spacelift.io/blog/terraform-test

Wang, Rosemary. HashiCorp. 2021. Testing HashiCorp Terraform[Webpage] Retrieved from https://www.hashicorp.com/blog/testing-hashicorp-terraform

---

Module 7: Ansible: configuration and orchestration

Ansible automates work on managed machines using readable playbooks. It is often used to install software, maintain configuration, deploy applications, and coordinate tasks across several hosts.

Expanded explanation

Control node and managed nodes

The control node runs Ansible. An inventory identifies managed hosts and groups them by role, such as web or database. A play selects a group and runs ordered tasks against it, commonly connecting through SSH for Linux hosts.

Tasks, modules, and playbooks

A task states an action through a module, such as ensuring that a package is installed or a file has a specified content. A playbook is a YAML document that groups tasks and declares which hosts receive them. Roles can package tasks, variables, templates, and handlers for reuse.

Idempotence

An idempotent task makes the necessary change once and then reports no change when the target already matches the desired state. This makes repeated runs safer. Modules that describe a state, such as present or absent, are usually preferable to ad hoc shell commands when a suitable module exists.

Handlers and variables

Variables let the same playbook work across environments. A handler runs when notified by a changed task, for example restarting a service only after its configuration file changed. This avoids needless disruption on every run.

Ansible alongside Terraform

Terraform commonly provisions cloud resources and tracks their lifecycle through state. Ansible commonly configures operating systems and applications on those resources. The boundary is not absolute, but assigning clear ownership prevents two tools from fighting over the same setting.

Practical example

A company has twenty web servers. An inventory groups them as web. One playbook installs the web server package, writes a configuration template, and ensures the service is running. When the template changes, a handler restarts the service; on the next unchanged run, the tasks report that the hosts are already in the desired state.

Key point

Ansible turns repeated machine setup into a readable, reusable process. Good playbooks describe desired outcomes and can be run safely again.

College lesson notes

Introduction

Module 7: Ansible - I

In this module, we will embark on a journey to explore Ansible's capabilities and learn how it can become an invaluable asset in your toolkit for managing and automating infrastructure. Whether you are a system administrator, DevOps engineer, or anyone interested in streamlining infrastructure operations, this module is designed to equip you with the knowledge and skills needed to harness Ansible's potential.

Learning Objectives

By the end of this module, you should be able to complete the following:

- Recognize the role of Ansible in validating infrastructure deployment scripts
- Construct automation scripts for scaling application components
- Explain Infrastructure as Code (IaC)
- Implement best practices for writing Ansible playbooks

Key Terms and Concepts

Key Term or Concept	Definition
CI/CD	Continuous Integration and Continuous Delivery (or Continuous Deployment).
IaC	Infrastructure as Code
SSH	Secure Shell, and it is a cryptographic network protocol used for secure communication over an unsecured network, such as the internet. SSH is widely used for secure remote access to systems, secure file transfers, and for configuring network devices and servers remotely.

Ansible

Ansible is an open-source automation tool used for configuration management, application deployment, task automation, and orchestration of infrastructure. It allows you to define and manage infrastructure and application configurations as code, making it easier to automate repetitive tasks and ensure consistency across your IT environment. Ansible is agentless, meaning it doesn't require software to be installed on managed nodes, making it lightweight and easy to set up. When we say that Ansible "doesn't require software to be installed on managed nodes," it means that Ansible can manage and automate remote servers or devices without needing any additional software or agent to be installed on those remote machines.

Key Features

The key features of Ansible include, but are not limited to:

- Agentless: Ansible doesn't require agents or software to be installed on managed nodes, reducing complexity and resource overhead.
- Idempotent: Ansible playbooks are idempotent, meaning they can be run multiple times without changing the system's state if the desired state has already been achieved. This reduces the risk of unintended changes.
- Declarative: Ansible playbooks use a declarative language, allowing you to define the desired state of your systems and infrastructure without specifying the exact steps to get there. Ansible figures out the most efficient way to achieve the desired state.
- Extensible: Ansible is highly extensible and supports custom modules, making it adaptable to various use cases and environments.

Using Ansible

Ansible is a great tool, and should be used for the following reasons:

- Automation: Ansible automates repetitive tasks, reducing human error and saving time.
- Consistency: It ensures that your infrastructure and configurations are consistent across all nodes, reducing configuration drift.
- Scalability: Ansible can easily scale to manage a large number of servers and configurations.
- Infrastructure as Code (IaC): Ansible enables IaC by allowing you to define infrastructure and configuration as code, which is versionable and can be collaboratively managed.

Differentiating Ansible from other Tools

Ansible is different from other IaC Tools in that it has the following:

- Agentless: Ansible doesn't require agents, unlike some other tools like Puppet or Chef.
- Simplicity: Ansible playbooks are often easier to write and read due to their declarative nature and YAML syntax.
- Ease of Learning: Ansible is relatively easy to learn and get started with, making it accessible to a wider audience.

The Need for Ansible

The need for Ansible arises from the growing complexity of modern IT environments. As organizations manage larger and more dynamic infrastructures, there's a greater demand for automation to streamline operations. Ansible addresses this need by providing a simple yet powerful way to automate tasks, enforce consistency, and embrace Infrastructure as Code principles, ultimately making infrastructure management more efficient and less error prone.

Real-world examples

Let’s take a moment to deconstruct Ansible to see how it applies to the real-world.

- 1. Configuration Management: Scenario: You have a fleet of web servers that need to have consistent configurations for security and performance. Ansible Use Case: You can use Ansible to write playbooks that define and enforce server configurations. For example, you can ensure that specific packages are installed, certain services are running, and firewall rules are correctly set across all servers in your infrastructure.
- 2. Application Deployment: Scenario: You have a web application that needs to be deployed to multiple servers whenever a new version is released. Ansible Use Case: Ansible can automate the deployment process. You can create playbooks that pull the latest code from a repository, configure the application, and restart services, ensuring a consistent and automated deployment across all servers.

- 3. Load Balancer Configuration: Scenario: You manage a group of load balancers, and you need to update the configuration to distribute traffic to newly added application servers. Ansible Use Case: Ansible can update the load balancer configuration dynamically. When new servers are added, Ansible can update the load balancer pool to include these servers, ensuring that traffic is distributed evenly without manual intervention.
- 4. Security Patching: Scenario: You need to regularly apply security patches to servers to protect against vulnerabilities. Ansible Use Case: Ansible can automate the process of patch management. You can create playbooks that connect to servers, check for available updates, and apply patches as needed. This ensures that all servers are up-to-date and secure.
- 5.Disaster Recovery: Scenario: In the event of a disaster, you need to quickly recover and restore your infrastructure and applications. Ansible Use Case: Ansible can help automate disaster recovery procedures. You can create playbooks that, when triggered, spin up backup servers, restore data from backups, and reconfigure the infrastructure to resume normal operations, reducing downtime and manual intervention.
- 6.Continuous Integration/Continuous Deployment (CI/CD): Scenario: You want to automate the testing and deployment of your software applications. Ansible Use Case: Ansible can be integrated into your CI/CD pipeline. You can use it to provision testing environments, deploy code changes, run tests, and promote successful builds to production, ensuring a streamlined and automated development process.

These real-world examples showcase the versatility of Ansible in automating various aspects of IT operations, from configuration management to deployment, security, and beyond. Ansible's ability to simplify complex tasks and enforce consistency is valuable in a wide range of industries and use cases.

Ansible Vs Terraform

Ansible and Terraform are both powerful automation tools, but they serve different purposes and have distinct characteristics. Below is a comparison of Ansible and Terraform to help you understand their differences and when to use each tool:

	Ansible	Terraform
Purpose	Ansible is primarily a configuration management and automation tool. It is used for tasks such as software installation, configuration, and orchestration. Ansible's main focus is on automating tasks on existing servers and managing configurations.	Terraform is an Infrastructure as Code (IaC) tool designed for provisioning and managing infrastructure resources. It's used to create, modify, and delete infrastructure components like virtual machines, networks, and databases. Terraform's primary purpose is to define and maintain infrastructure resources in a declarative manner.
Use-Cases	Configuration management of existing servers and systems. Application deployment and orchestration. Task automation, including software updates, user management, and backups.	Infrastructure provisioning in various cloud providers (AWS, Azure, Google Cloud, etc.). Managing infrastructure resources across multiple cloud platforms. Building, scaling, and tearing down infrastructure environments.
Languages and Syntax	Ansible uses YAML for defining tasks and playbooks. It's human-readable and relatively easy to learn.	Terraform uses its own HashiCorp Configuration Language (HCL). While HCL is designed for describing infrastructure, it may require some additional learning compared to YAML.
State Management	Ansible is stateless by design. It doesn't inherently maintain a state file to track infrastructure changes.	Terraform maintains a state file that keeps track of the current infrastructure state. This allows Terraform to plan and apply changes incrementally.
Modularity and Reusability	Ansible uses playbooks, roles, and modules for organizing and reusing automation code. Playbooks can contain tasks for various purposes.	Terraform uses modules for organizing and reusing infrastructure code. Modules are designed to encapsulate and reuse infrastructure components.

	Ansible	Terraform
Agent vs. Agentless	Ansible is agentless and relies on SSH or other remote access methods to execute tasks on managed nodes.	Terraform doesn't interact directly with managed nodes but instead communicates with APIs provided by cloud providers or infrastructure platforms.
Learning Curve	Ansible is relatively easy to learn, especially for those familiar with YAML and basic scripting.	Terraform may have a steeper learning curve due to its unique HCL syntax and the need to understand the intricacies of different cloud provider APIs.
Complementary Tools	Often used in conjunction with Terraform to configure provisioned infrastructure after it's created.	Works well with Ansible for tasks like post-provisioning configuration management.

In summary, Ansible and Terraform are not direct competitors but are often used together to complement each other's strengths. Ansible is excellent for configuration management and task automation on existing systems, while Terraform shines in provisioning and managing infrastructure resources in a declarative, Infrastructure as Code manner. The choice between them depends on your specific automation needs and the scope of your infrastructure management tasks.

Fundamental concepts in Ansible

Let’s shift our attention back to Ansible, by breaking down it’s fundamental concepts.



Ansible tasks: Actions that Ansible will execute.

Ansible Inventory: A list of hosts where ansible will execute tasks.

Ansible play: A mappying between groups of hosts in the inventory and tasks to perform.

Ansible playbook: A file with all the plaus that ansible will execute over the inventiry. It’s the definition of the state you want the system to have.

Let’s take a look at these fundamental concepts in greater detail.

1. Ansible Tasks:

- Definition: Ansible tasks are individual actions or operations that Ansible performs on the target hosts. They are the building blocks of Ansible playbooks.
- Idempotence: As mentioned, Ansible tasks are designed to be idempotent, meaning running the same task multiple times won't change the system's state after the initial run. This ensures that tasks are safe to run repeatedly.
- Examples: Ansible tasks can include installing packages, creating directories, copying files, restarting services, and any other action you want to automate on remote hosts.

2. Ansible Inventory:

- Definition: Ansible inventory is a list of hosts and groups of hosts that Ansible will manage. It specifies which machines Ansible will connect to and apply tasks.
- Sources: Inventory can be defined in static inventory files or generated dynamically from various sources, such as cloud providers, database queries, or custom scripts.

- Examples: In an inventory, you can define hosts by their IP addresses or domain names and organize them into groups for easier management.

This is what a simple Ansible inventory file looks like: What you're looking at is basically a list of computer addresses. These can be either IP addresses (like 192.168.1.23) or website names (like server1.domain.com). But it's not limited to just that; you can also use things like address ranges or even create custom instructions to find all the computers where your automation will happen.

```
192.168.1.23
[db]
10.2.14.23
[webservers]
server1.domain.com
server2.domain.org
```

Additionally, the text you see in square brackets [ ] is used to group these computers together. When you're running your automation, and it mentions a group, Ansible will only connect to and work on the computers that belong to that group. So, it's a way to organize and manage which computers are involved in each task.

3. Ansible Play:

- Definition: An Ansible play is a section within an Ansible playbook. It defines a set of tasks that should be executed on a specific group of hosts defined in the inventory.
- Role in Playbook: Plays map hosts to tasks. Each play is associated with one or more hosts or groups of hosts and specifies which tasks should be run on those hosts.
- Examples: In a playbook, you might have a play that targets web servers and another play that targets database servers, each with its set of tasks.

4. Ansible Playbook:

- Definition: An Ansible playbook is a YAML file that contains a collection of plays. It defines the desired state of the system and the automation tasks required to achieve that state.
- Composition: Playbooks are used to orchestrate the execution of tasks across multiple hosts and groups. They allow you to define the order in which tasks should be executed.
- Examples: A playbook can include plays for setting up a web server, configuring a database, and deploying a web application.

This is a playbook example from Ansible documentation:

```
---
- hosts: webservers
remote_user: yourname
tasks:
- service:
name: nginx
state: started
become: yes
become_method: sudo
```

The playbook is like a set of instructions for Ansible. It says, "Hey Ansible, go to all the computers in the 'webservers' group and use this user to connect to them." Then, it asks Ansible to act like a superuser (with special powers) so it can check if the web server called Nginx is running. If Nginx is not running, the playbook tells Ansible to start it. If Nginx is already running, the playbook doesn't do anything.The playbook is written in a format called YAML, which is very easy to read. This means that everyone on your team can look at it and understand what's happening with the tasks that Ansible is doing.

So, now that we have our list of computers (inventory) and our set of instructions (playbook), it's time to make Ansible do its thing. And here's how we do it:

- You open up your computer's command line or terminal, and you type in this command: $ ansible-playbook playbook.yml.
- When you hit enter, Ansible will start working its magic. It will connect to all the computers in the 'webservers' group, and like a wizard, it will make sure that the Nginx web service is up and running on each of them. It's almost like a magical spell!

These concepts are foundational to understanding how Ansible works and how you can use it to automate tasks, manage infrastructure, and achieve desired system states in a safe and efficient manner.

Summary

Conclusion

In this module we learned that Ansible helps automate tasks like configuring servers, deploying applications, and managing infrastructure. It follows the concept of Infrastructure as Code (IaC), where you define and manage infrastructure using code. Key components include playbooks (sets of automation instructions), inventory (lists of managed hosts), and modules (individual automation actions). Ansible is agentless, meaning it doesn't require additional software on managed nodes.

References

Intellipaat. 2023. Terraform vs. Ansible - Difference [Webpage] Retrieved from https://intellipaat.com/blog/terraform-vs-ansible-difference/

RedHat. 2022. Learning Ansible Tutorial [Webpage] Retrieved from https://www.redhat.com/en/topics/automation/learning-ansible-tutorial

Sequeira, Anthony. LinkedIn Learning. 2020. Learning ansible. [Video] Retrieved from Starting your Ansible journey (linkedin.com)

Spaleta, Jef. Sensu. 2020. Implementing Infrastructure as Code with Ansible [Webapge] Retrieved from https://sensu.io/blog/implementing-infrastructure-as-code-with-ansible#:~:text=A%20basic%20IaC%20workflow%20with%20Ansible.,-What%20is%20Ansible&text=In%20very%20simple%20words%2C%20Ansible,up%20a%20full%20deployment%20orchestration.

Module 8: Ansible: execution and YAML

To use Ansible well, understand how the control node selects hosts, connects to them, runs modules, and interprets YAML. Many playbook problems come from a wrong host group, connection assumption, or indentation level rather than a complicated algorithm.

Expanded explanation

Execution path

Ansible reads the inventory, selects hosts in the play, loads variables, and executes tasks in order. Tasks usually run across all selected hosts before the next task begins. A failure should be investigated at the task and host where it occurred.

YAML structure

YAML uses indentation to represent nesting. A dash starts a list item; a colon separates a key from its value. Tabs, inconsistent indentation, or an accidental list level can change the meaning of a playbook. Quotes help when a value could be interpreted as another type or contains special characters.

Inventory and host groups

An inventory can list individual hosts and logical groups. Plays target a group name, making it possible to configure web servers differently from database servers. Variables can be scoped to groups or hosts, but excessive overrides make behavior harder to predict.

Privilege and connectivity

The control node needs a working connection and suitable permissions on managed nodes. Privilege escalation is requested for tasks that need it, rather than assuming every action runs as an administrator. Test access on a small host set before a broad run.

Safe execution

A syntax check catches malformed YAML or playbook structure. Check mode previews many changes but cannot perfectly simulate every module. For risky changes, use a staging group or limited host batch and verify the resulting service behavior.

Illustrative example

```
- name: Configure web servers
  hosts: web
  become: true
  tasks:
    - name: Ensure the web package is installed
      ansible.builtin.package:
        name: nginx
        state: present
```

Practical example

The play targets hosts in the web group. The task asks the package module to ensure nginx is present. The indentation makes the package arguments children of the module name, and the task itself a member of the tasks list. Re-running it should not reinstall an already installed package.

Key point

Read an Ansible playbook as a hierarchy: which hosts, which privileges, which ordered tasks, and which desired state each task enforces.

College lesson notes

Introduction

Module 8: Ansible - II

In the previous module you explored Ansible's capabilities and learned how it can become an invaluable asset in your toolkit for managing and automating infrastructure. This module will expand on that foundational knowledge, by examining Ansible's core concepts, how it interacts with control and remote machines, and the basics of YAML syntax. This knowledge will solidify your understanding of Ansible so that you can apply it to the world of automation.

Learning Objectives

By the end of this module, you should be able to complete the following:

- Recognize how Ansible works
- Outline the different types of machines
- Explain the installation process
- Explain the basics of YAML

Key Terms and Concepts

Key Term or Concept	Definition
CI/CD	Continuous Integration and Continuous Delivery (or Continuous Deployment).
IaC	Infrastructure as Code
SSH	Secure Shell, and it is a cryptographic network protocol used for secure communication over an unsecured network, such as the internet. SSH is widely used for secure remote access to systems, secure file transfers, and for configuring network devices and servers remotely.
YAML (Yet Another Markup Language)	A human-readable data serialization language that is often used for writing configuration files

How Ansible Works

Recall, that Ansible is a powerful automation tool used to manage and configure multiple remote machines, making system administration tasks more efficient. It does this through a simple and agentless approach, which means you don't need to install any additional software on the target machines.

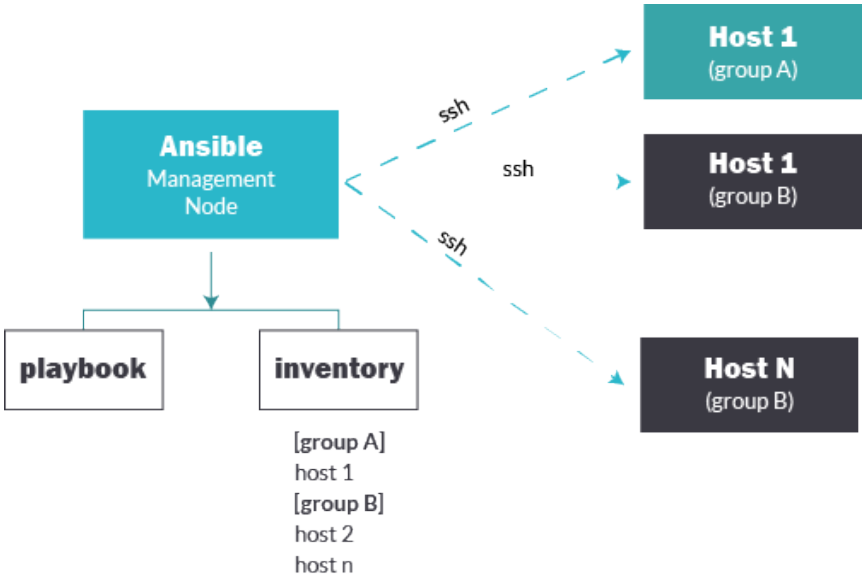
Components of Ansible

Ansible is made up of a few different components:

- Controlling Node (Management Node): This is the machine where you have Ansible installed and run your commands. The controlling node acts as the central command center for Ansible.
- Inventory: An inventory file contains a list of the hosts or remote machines where you want to execute tasks. These hosts can be grouped into categories, making it easier to manage and apply configurations to different sets of machines.
- Ansible Modules: Ansible modules are small programs or scripts that perform specific tasks on remote hosts. These modules are responsible for carrying out actions like installing software, configuring services, or managing files on the remote machines.

How Ansible Works

Ansible follows a straightforward workflow, as illustrated below: Let’s deconstruct this workflow.



- Inventory Setup: You create an inventory file that lists the remote hosts you want to manage. Here's a simple example of an inventory file:[web_servers]server1 ansible_host=192.168.1.10 ansible_user=myuser ansible_ssh_pass=mypasswordserver2 ansible_host=192.168.1.11 ansible_user=myuser ansible_ssh_pass=mypassword

In this example, we have two remote servers, server1 and server2, along with their IP addresses and login credentials.

- Playbook Creation: Ansible uses playbooks, which are YAML files that describe the tasks to be executed on the remote hosts. Here's a basic playbook example:

- name: Install Apache hosts: web_servers tasks: - name: Install Apache apt: name: apache2 state: present

In this playbook, we specify that we want to install Apache on the hosts listed under the web_servers group.

- Running Ansible: You run Ansible from the controlling node and specify the playbook you want to execute:

ansible-playbook myplaybook.yml

- Module Execution: Ansible connects to the remote hosts via SSH (by default) and pushes the necessary Ansible modules to them. In our case, it will send a module to install Apache.
- Result Reporting: Ansible executes the tasks on the remote hosts and provides real-time feedback on whether they were successful or not. You can view the results and troubleshoot any issues that may arise.
- Cleanup: One of the beauties of Ansible is that it automatically removes the modules it pushed to the remote hosts after the tasks are completed successfully. This means it cleans up any temporary files or scripts it used during the execution.

Types of machines

In an Ansible deployment, there are two main types of machines:

- Control Machines; and
- Remote machines

Control Machines:

- The control machine is the central system from which you manage and control other remote machines.
- Ansible is installed and configured on the control machine.
- It's the machine where you write Ansible playbooks, execute commands, and manage the entire automation process.
- The control machine communicates with remote machines and sends Ansible modules to them for execution.
- It acts as the brain behind the automation, orchestrating tasks and ensuring consistency across remote machines.

Remote Machines:

- Remote machines are the target systems that you want to manage, configure, or automate.
- Ansible connects to these remote machines via SSH (by default) or other supported transport methods.
- Multiple remote machines can be managed by a single control machine.
- Remote machines can have different operating systems and configurations, but Ansible abstracts the differences, allowing you to manage them uniformly.
- Ansible modules are executed on these remote machines to perform tasks such as software installation, configuration changes, and more.

In summary, the control machine is where Ansible is installed and used to orchestrate actions on one or more remote machines. Ansible's strength lies in its ability to efficiently manage and automate tasks across a wide range of remote systems, making it a valuable tool for configuration management, software deployment, and system administration.

Installation Process

Now that you have a good understanding of how Ansible works and the types of machines, let’s examine the Control Machine’s Requirements for Ansible:

- Python Compatibility: Ansible can be run on a machine with either Python 2 (versions 2.6 or 2.7) or Python 3 (versions 3.5 and higher) installed. Python is a fundamental requirement because Ansible itself is written in Python.
- Windows Limitation: Ansible is primarily designed to run on Unix-like systems (Linux, macOS, etc.). It is not supported natively on Windows. However, you can still use Ansible to manage Windows machines as remote hosts, but you need to run Ansible on a compatible control machine or use Windows Subsystem for Linux (WSL) to execute Ansible playbooks.
- SSH Communication: By default, Ansible uses SSH (Secure Shell) to establish communication with and manage remote machines. This means that the remote machines you want to manage should have SSH access enabled.

- No Database or Daemons: Ansible is agentless and does not require any databases or daemons to be running on the control machine. It operates as a standalone tool, making it lightweight and easy to set up.
- No Residue on Remote Machines: Ansible connects to remote machines, performs tasks, and then removes any temporary files or scripts it used during execution. It leaves no software or agents running on the remote machines, ensuring a clean and non-intrusive approach to automation.
- Installation Methods: Ansible can be installed on a control machine in various ways, depending on the operating system. Common package managers like Apt, yum, pkg, and pacman can be used, or you can install Ansible using Python's package manager, pip. The installation method may vary depending on your specific environment and requirements.

Ansible on an Ubuntu Machine

To install Ansible on an Ubuntu machine using the apt package manager, the following steps are important as to ensure Ansible is properly configured and installed. Here's a breakdown of the commands.

- Update Package Lists: $ sudo apt-get update

This command updates the package lists on your Ubuntu system, ensuring that you have the latest information about available packages.

- Install software-properties-common:$ sudo apt-get install software-properties-common

This command installs the software-properties-common package, which provides essential tools for managing software repositories.

- Add Ansible PPA (Personal Package Archive):$ sudo apt-add-repository ppa:ansible/ansible

This command adds the Ansible PPA to your list of software sources, allowing you to install Ansible from the PPA.

- Update Package Lists (Again):$ sudo apt-get update

After adding the Ansible PPA, you should update your package lists again to include the newly added repository.

- Install Ansible:$ sudo apt-get install ansible

This command installs Ansible on your Ubuntu machine.

- Check Ansible Version:$ ansible -version

You can use this command to verify that Ansible was installed successfully and to check the installed version.

The above steps ensures that Ansible is properly set up on your Ubuntu machine, and you can start managing remote machines using Ansible playbooks and commands.

YAML Basics

Earlier in the module we touched on an important term called YAML. Let’s take a moment to quickly introduce it’s purpose and use with Ansible.

YAML is a human-readable data serialization language that is often used for writing configuration files. It is a popular programming language because it is designed to be easy to read and understand and can also be used in conjunction with other programming languages. Because of its flexibility and accessibility, YAML is used by the Ansible automation tool to create automation processed in the form of Ansible Playbooks.

The basics of YAML are as follows:

- YAML files often start with --- and end with ..., although these are optional markers.
- YAML uses a key-value pair format to represent data, with a colon (:) and space separating the key and value.
- Proper indentation is crucial in YAML to denote nested structures.

Dictionary (Key-Value Pairs):

- Dictionaries are represented in YAML using key-value pairs.
- Example:

```yaml
james:
  name: james john
  rollNo: 34
  div: B
  sex: male
```

Abbreviation for Dictionaries:

- Dictionaries can also be abbreviated using curly braces.
- Example:

```yaml
James: {name: james john, rollNo: 34, div: B, sex: male}
```

Lists:

- Lists are represented in YAML by placing each element on a new line with the same indentation and preceded by a dash and space.
- Example:

```yaml
countries:
  - America
  - China
  - Canada
  - Iceland
```

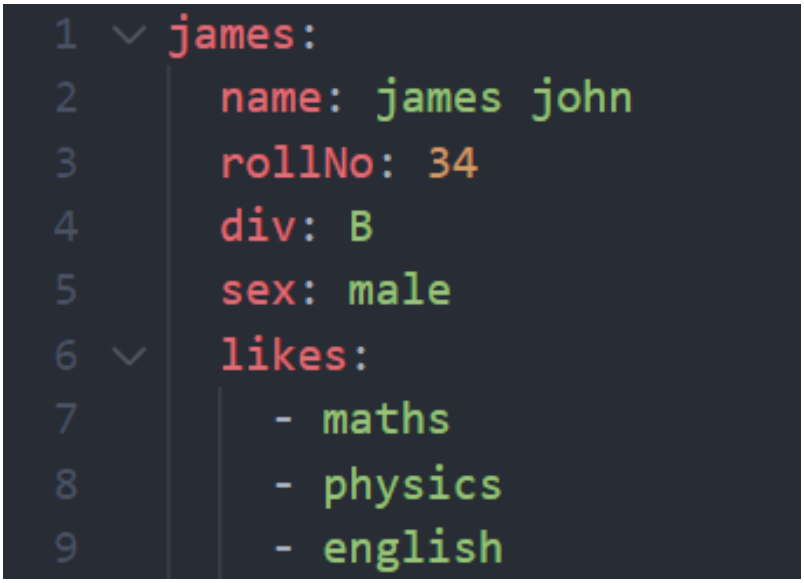
Abbreviation for Lists:

- Lists can also be abbreviated using square brackets.
- Example:

```yaml
Countries: ['America', 'China', 'Canada', 'Iceland']
```

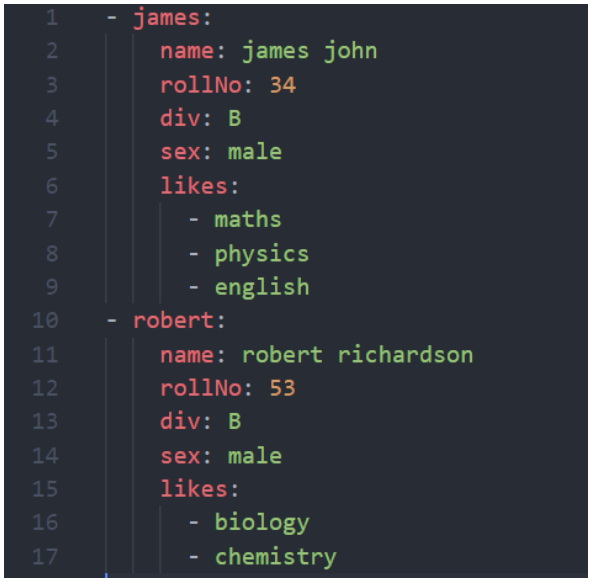
Lists Inside Dictionaries:

- You can include lists as values within dictionaries.
- Example:



List of Dictionaries:

- You can create a list of dictionaries.
- Example:



Boolean Values:

YAML supports Boolean values (true/false), and these values can be case-insensitive.

Example passed: TRUE

YAML's flexibility and readability make it a suitable choice for creating Ansible playbooks, where complex configurations and data need to be expressed in a clear and concise format.

Summary

Conclusion

In summary, Ansible is an efficient automation tool that embraces IaC principles, allowing you to define and manage infrastructure through code. It's driven by components like playbooks, inventory, and modules, and it excels with its agentless approach, making it an asset for automating diverse tasks in IT operations.

References

IntelliPaat Software Solutions. 2023. Terraform vs. Ansible: Key differences among terraform and Ansible. [Webpage] Retrieved from https://intellipaat.com/blog/terraform-vs-ansible-difference/

RedHat Inc. 2023. Learning Ansible Basics. [Webpage] Retrieved from https://www.redhat.com/en/topics/automation/learning-ansible-tutorial

Rising Stack. 2022. Getting Started with Ansible Infrastructure Automation. [Webpage] Retrieved from https://blog.risingstack.com/getting-started-with-ansible-infrastructure-automation/#creatinganansibleplaybook

Sequeira, Anthony. LinkedIn Learning. 2020. Learning ansible. [Video] Retrieved from Starting your Ansible journey (linkedin.com)

Spaleta, Jef. Sensy by Sumo Logic. 2020. Implementing Infrastructure as Code with Ansible. [Webpage] Retrieved from https://sensu.io/blog/implementing-infrastructure-as-code-with-ansible#:~:text=A%20basic%20IaC%20workflow%20with%20Ansible.,-What%20is%20Ansible&text=In%20very%20simple%20words%2C%20Ansible,up%20a%20full%20deployment%20orchestration.

Tutorialspoint. 2023. Ansible Playbooks Tutorial. [Webpage] Retrieved from https://www.tutorialspoint.com/ansible/ansible_playbooks.htm

---

Module 9: Azure Bicep: resource definitions

Bicep is a concise language for describing Azure resources. It compiles to Azure Resource Manager (ARM) templates, so the Azure deployment system still performs the resource operations while authors work with a more readable syntax.

Expanded explanation

Bicep and ARM templates

ARM templates express Azure deployments in JSON. Bicep describes the same deployment model with fewer structural characters and clearer symbolic references. The generated ARM JSON is an implementation detail for most authors, although understanding the resource graph remains important.

Resources and dependencies

A resource block states the type, API version, name, location when required, and properties of an Azure resource. A reference to another resource can establish a dependency. Explicit dependencies are available when a relationship is not expressed by a direct reference.

Modules

A Bicep module packages a reusable deployment unit. A parent file can pass parameters to the module and consume outputs. Keep modules focused on a meaningful responsibility, such as a network or application service, and make the required inputs clear.

Bicep versus Terraform

Bicep is designed for Azure deployments and uses Azure Resource Manager as its deployment engine. Terraform manages resources through providers and uses a state file to map configuration to objects. The best choice depends on the environment, existing workflow, team expertise, and need to manage services outside Azure.

State and drift

Terraform explicitly stores state. Bicep does not require a Terraform-style local state file for resource mapping; Azure retains the resource state. That difference does not eliminate drift: manual Azure changes can still make the deployed environment differ from the Bicep file, so review and what-if checks remain useful.

Practical example

A Bicep file declares a virtual network and a subnet that refers to it. Azure Resource Manager can infer that the network must exist before the subnet is created. The file can be reviewed and redeployed for another environment by supplying different names and address ranges as parameters.

Key point

Bicep makes Azure resource definitions easier to read while keeping Azure Resource Manager responsible for deployment. Think in terms of resources, inputs, outputs, and dependencies.

College lesson notes

Introduction

Module 9 : Introduction to Bicep- I

In this module, we will dive deep into the world of Azure Bicep, an innovative Infrastructure as Code (IaC) solution provided by Microsoft. As technology continues to evolve, the need for efficient, scalable, and automated infrastructure deployment becomes paramount. Azure Bicep empowers engineers, developers, and DevOps professionals to achieve just that, streamlining the process of provisioning and managing cloud resources.

This module will provide you with a comprehensive understanding of Bicep's structure, syntax, and practical applications. We'll explore various facets of Bicep, from basic concepts to advanced techniques, while highlighting its importance in the context of modern cloud infrastructure.

Learning Objectives

By the end of this module, you should be able to complete the following:

- Explain the premise of Bicep
- Compare and contrast Bicep with Terraform
- Outline the benefits of Azure Bicep

Key Terms and Concepts

Key Term or Concept	Definition
ARM	Azure Resource Manager
CI/CD	Continuous Integration and Continuous Delivery (or Continuous Deployment).
IaC	Infrastructure as Code

Azure Bicep

Azure Bicep is a game-changing domain-specific language designed to simplify and enhance the process of building and deploying Azure resources. At its core, Bicep serves as an abstraction layer, offering a more user-friendly and efficient approach compared to the traditional Azure Resource Manager (ARM) templates. These templates, often written in JSON, can become intricate and convoluted, particularly in the context of extensive deployments.

- Azure Bicep is a fresh way to describe how you want your Azure resources to be set up. It's like giving instructions to build your cloud-based things. But, Bicep is not a universal language. It's made by Microsoft for working only with Azure.
- Imagine Bicep as a special tool just for Azure, a bit like a custom recipe book for cooking in a specific kitchen. You can't use this tool to cook things in other kitchens (cloud providers) or to make completely different things (non-Azure resources).
- Just remember, Bicep isn't for writing general computer programs like games or apps. It's all about easily getting your Azure stuff done.
- Azure Bicep makes building Azure resources easier by providing simpler syntax, improved code reuse, and flexible file organization. It excels in modularity and code reusability, enhancing the overall authoring experience.

Comparing Azure Bicep and Azure ARM Templates

Azure Resource Manager (ARM) templates serve as representations of Azure resources, scripted in a specialized JSON format that goes beyond typical JSON. This syntax incorporates functions and methods to handle intricate tasks. With that, however, working with ARM JSON can be challenging due to its complex syntax. Bicep addresses this challenge with a simplified language approach. When creating an ARM template, intricate expressions are necessary, often leading to verbose outputs. Let's contrast the syntax and code needed to deploy an Azure Virtual Network using ARM JSON versus Bicep syntax. Here's a basic ARM template for the task, spanning 27 lines:

1. ARM JSON

```
{
  "$schema": "https://schema.management.azure.com/schemas/2019-04-01/deploymentTemplate.json#",
  "contentVersion": "1.0.0.0",
  "resources": [
    {
      1 child: snet1 (subnets)
      "type": "Microsoft.Network/virtualNetworks",
      "apiVersion": "2020-11-01",
      "name": "vnet-westus-001",
      "location": "[resourceGroup().location]",
      "properties": {
        "addressSpace": {
          "addressPrefixes": [
            "10.10.0.0/16"
          ]
        },
        "subnets": [
          {
            Parent: vnet-westus-001 (virtualNetworks)
            "name": "snet1",
            "properties": {
              "addressPrefix": "10.10.5.0/24"
            }
          }
        ]
      }
    }
  ]
}
```

2. Azure Bicep

In contrast, achieving the same resource deployment with a Bicep template requires just 19 lines. At this moment understanding the details of the Bicep syntax is not required; we'll explain more about how Bicep files work as you continue with this Module.

```
resource myVnet 'Microsoft.Network/virtualNetworks@2020-11-01' = {
  name: 'vnet-westus-001'
  location: resourceGroup().location
  properties: {
    addressSpace: {
      addressPrefixes: [
        '10.10.0.0/16'
      ]
    }
    subnets: [
      {
        name: 'snet1'
        properties: {
          addressPrefix: '10.10.5.0/24'
        }
      }
    ]
  }
}
```

Benefits of Azure Bicep

Some of the benefits of using Bicep include:

- Easier Syntax
- Modules
- Managing dependencies.

Let’s take a closer look at each of these benefits.

Easier SyntaxBicep uses a simpler syntax for creating templates. Recall from modules 5 and 6 where you learned about HashiCorp Terraform, learning Bicep is easy as their syntax is quite similar. Bicep allows you to reference parameters and variables without using complex functions. It also uses string interpolation, which is simpler than joining strings together. Additionally, you can refer to resource properties using their names instead of complicated references. This makes Bicep templates easier to write and understand compared to ARM JSON syntax.

ModulesSecondly, Bicep allows you to build complex deployments by making smaller module files. You can then use these modules in your main template. For example, you could have a module that creates a virtual machine, and then you just refer to it from your main template. This modular approach helps keep your code organized and makes it easier to manage.

Managing Dependencies Lastly, with Bicep, you often don't need to worry about resource dependencies. In ARM templates, you must specify these dependencies yourself. For instance, creating a virtual machine requires a network interface, which needs a virtual network and subnet. Bicep handles these resource dependencies automatically. It figures out the order in which things need to be deployed, so you don't have to do it manually.

Bicep Vs Terraform

HashiCorp Terraform and Azure Bicep are both tools for managing Microsoft Azure infrastructure using Infrastructure as Code (IaC). They have slightly different philosophies of use. Terraform can target multiple cloud providers, while Azure Bicep is focused solely on managing Azure resources.

Terraform Management	Azure Bicep Management
Works across on-premises and multiple cloud providers. Uses HashiCorp Configuration Language (HCL) for infrastructure configuration. Supports versioning, automation, and CI/CD integration. Supports Azure through a provider model. Provides benefits like declarative IaC, version control, and multi-cloud support.	Built natively for Azure IaC. Transpiles code into ARM JSON before deployment. Offers the same IaC features as ARM. Provides readable and maintainable code. Supports declarative IaC, version control, and simplified syntax.

When distinguishing between the two management types, consider the following:

Choosing the Best Tool:

- Both tools have benefits.
- Azure Bicep is integrated with Azure and transpiles to ARM.
- Terraform is more versatile, supports multi-cloud, and uses a provider model.
- Consider organizational needs and requirements when choosing.

When to Choose Azure Bicep:

- Best for Azure-focused projects.
- Seamless Azure integration and support.
- Trust in Microsoft's support and future development.

When to Choose HashiCorp Terraform:

- Best for multi-cloud or hybrid environments.
- Supports various clouds and platforms.
- Can manage diverse infrastructure needs.
- Offers flexibility and community support.

Conclusion:

- Both tools have strengths.
- Consider project and organizational requirements.
- Knowledge of both tools is valuable for Software developers and DevOps engineers.
- Remember, choosing between Terraform and Azure Bicep depends on your specific project needs and environment. It's essential to evaluate the features and trade-offs to make the best decision for your organization.

Differentiating between Terraform State and Azure Bicep

In Terraform, a state file is a crucial component that keeps track of the current state of your deployed infrastructure. It records the actual resources that have been created, their configurations, and their dependencies. This state file allows Terraform to understand what resources are currently deployed and enables it to make changes or updates in a safe and controlled manner.

Here's a simple example to illustrate the concept of a state file:

Example

Let's say you're using Terraform to deploy a virtual machine (VM) in Azure. When you run terraform apply, Terraform will create the VM according to the configuration you've provided in your .tf files. After the deployment, Terraform generates a state file that includes information about the VM, such as its location, size, and other settings.

Now, let's say you decide to update the VM's configuration by changing its size in your .tf files. When you run terraform apply again, Terraform will analyze the state file to determine what changes need to be made. It will then update the VM's size accordingly, ensuring that the actual state matches the desired configuration.

The state file essentially acts as a "source of truth" for your infrastructure. It prevents unintended changes and helps Terraform maintain the desired state of your resources.

Differentiation

Azure Bicep, being part of Microsoft's native toolset, manages the state in a different way compared to Terraform. In Azure Bicep, there's no explicit state file that you need to manage. Instead, Azure Bicep relies on incremental deployment. When you deploy an Azure Bicep template, it analyzes the existing resources in your Azure subscription and deploys only the changes or additions. This means that Azure Bicep doesn't need to store a separate state file; it directly communicates with the Azure Resource Manager to manage the state.

In summary, Terraform uses a state file to keep track of the deployed resources and their configurations.Azure Bicep uses incremental deployment and directly communicates with Azure Resource Manager to manage the state without a separate state file.

Summary

Conclusion

In this module, you differentiated between two prominent Infrastructure as Code (IaC) tools: Terraform and Azure Bicep. You learned that Terraform is a versatile, cross-cloud IaC tool that supports multiple providers, while Azure Bicep is designed specifically for Microsoft Azure. Both tools offer declarative approaches to provisioning and managing cloud infrastructure, with key differences in their philosophies and integrations. Additionally, Terraform provides extensive multi-cloud support and uses HashiCorp Configuration Language (HCL), while Azure Bicep is closely integrated with Azure, offering a more readable language and transpiling to Azure Resource Manager (ARM) templates. This module covered the benefits, definitions of Bicep, and presented a comprehensive overview, enabling you to make informed decisions when selecting the most suitable tool for their IaC needs.

References

Brown, Jeff. 4sysops. 2021. Azure Bicep: Getting Started guide. [Webpage] Retrieved from https://4sysops.com/archives/azure-bicep-getting-started-guide/

Brown, Jeff. Varonis. 2022. Azure Bicep: Getting Started and How-to-Guide. [Webpage] Retrieved from https://www.varonis.com/blog/azure-bicep

Lean Sarah. Techielass. 2021. Getting Started with Azure Bicep. [webpage] Retrieved from https://www.techielass.com/getting-started-with-azure-bicep/

Pietschmann, Chris. Build5Nines. 2023. Comparison: Terraform vs Azure Bicep. [Webpage] Retrieved from https://build5nines.com/comparison-terraform-vs-azure-bicep

Yusubov, Elkhan. LinkedInLearning. 2021. Azure Bicep First Look. [Video] Retrieved from Learning Azure Bicep (linkedin.com)

Module 10: Azure Bicep: authoring and deployment

A useful Bicep file separates environment-specific inputs from reusable logic. Parameters accept deployment values, variables name derived expressions, resources declare Azure objects, and outputs expose selected results.

Expanded explanation

Parameters versus variables

A parameter is supplied by the caller or has a default. Use it for a choice that changes between deployments, such as location or environment name. A variable is computed inside the template and is useful for repeated expressions. Do not turn every literal into a parameter; expose only decisions the caller truly needs.

Resource properties and references

A resource declaration needs the correct type and API version and the properties required for that service. Symbolic names let one resource refer to another without hard-coding an ID. Outputs should expose useful values, not secrets.

Deployment scope

A deployment runs at a scope such as a resource group, subscription, management group, or tenant, depending on the file and command. Scope affects which resources can be created and which permissions are needed. Confirm the target subscription and resource group before running a deployment.

Preview and verification

Validate the file, use a what-if preview to understand the expected changes, and then deploy through an approved workflow. After deployment, verify the resources and application behavior. A successful template operation does not prove that every service is configured for its intended workload.

Reusable templates

Use modules for repeated deployment patterns and parameter files for environment-specific values. Keep names and outputs understandable, and record why an unusual configuration is needed. Review changes in version control just as you would application code.

Illustrative example

```
param location string = resourceGroup().location
param storageName string

resource storage 'Microsoft.Storage/storageAccounts@2023-05-01' = {
  name: storageName
  location: location
  sku: { name: 'Standard_LRS' }
  kind: 'StorageV2'
}

output storageId string = storage.id
```

Practical example

This illustrative Bicep file takes a storage account name as input and defaults its location to the current resource group. The resource block declares the account, and the output exposes its ID. Before use, check that the chosen name is valid and available, that the API version and properties meet the current Azure requirements, and that the target scope is correct.

Key point

A strong Bicep workflow moves from clear inputs to reviewed resource definitions, a preview of changes, controlled deployment, and post-deployment verification.

College lesson notes

Introduction

Module 10 : Introduction to Bicep- II

Welcome to Module 10.This final module is the second part to Bicep, and will expand on your previously built knowledge by having you apply key concepts. Here you will learn to write infrastructure deployment scripts and construct automation scripts for scaling application components using Bicep.

Learning Objectives

By the end of this module, you should be able to complete the following:

- Create Infrastructure Deployment Scripts Using Bicep
- Validate Infrastructure Deployment Scripts with Bicep
- Construct Automation Scripts for Scaling Application Components Using Bicep

Key Terms and Concepts

Key Term or Concept	Definition
CI/CD	Continuous Integration and Continuous Delivery (or Continuous Deployment).
IaC	Infrastructure as Code

Prerequisite

To work with Azure Bicep and deploy Bicep files, you'll need to set up the necessary tools on your local system. Microsoft offers various installation options for Windows, Linux, and macOS platforms.

To kickstart your journey with Azure Bicep, ensure you have the following components set up on your local system:

- Visual Studio Code with the Bicep extension; and either
- Azure CLI or Azure PowerShell.

These tools are essential for crafting, visualizing, and deploying your resources using an Azure Bicep template. Here’s a little more information on AzureCLI and Azure PowerShell.

Azure CLI

The Azure CLI is a set of commands for creating and managing Azure resources. Starting from Azure CLI version 2.20.0 or later, the Bicep CLI binaries are automatically included. This means you don't need to install Bicep separately when using it with the Azure CLI. The Azure CLI natively supports deploying Bicep files using deployment commands. For instance, you can directly deploy a Bicep file using the az deployment group create command. The Azure CLI also supports various Bicep-specific commands, accessible through az bicep.

Azure PowerShell

Azure PowerShell is a module for deploying and managing Azure resources using PowerShell. While it doesn't automatically include Bicep CLI commands, it works with Bicep CLI at version 5.6.0 or later. You'll need to install Bicep CLI separately and add it to your PATH environment variable. Keep an eye out for future versions of Azure PowerShell that might natively support Bicep CLI commands.

Use the following resources to help get you started.

- Visual Studio Code

- Bicep extension
- Azure CLI or Azure PowerShell,

Writing Azure Bicep files

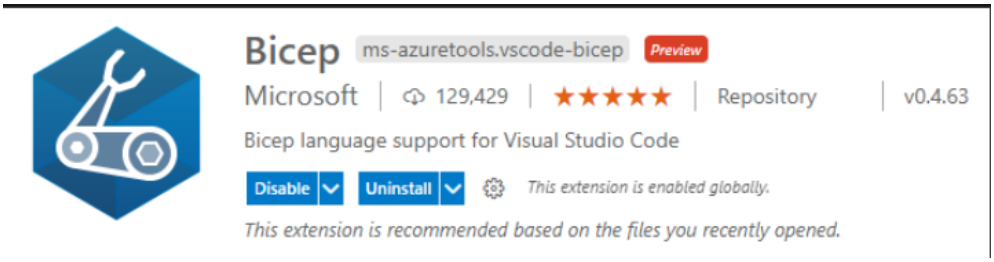
Creating Bicep templates is versatile and can be done using your preferred tool or integrated development environment (IDE). Bicep templates are saved with a .bicep extension. For an optimal authoring experience, consider leveraging Visual Studio Code, which is highly recommended. Microsoft offers a dedicated Bicep extension for Visual Studio Code, enhancing your template creation process with language support and auto-completion features to streamline the composition of Bicep templates.

To work with Bicep files, you need a good editor. It is suggested to use VSCode:

- Visual Studio Code: If you don't have it, install Visual Studio Code on your computer.
- Bicep Extension for Visual Studio Code: This is like an add-on that makes Visual Studio Code even better for working with Bicep. It helps you write Bicep files correctly and quickly.

Installation

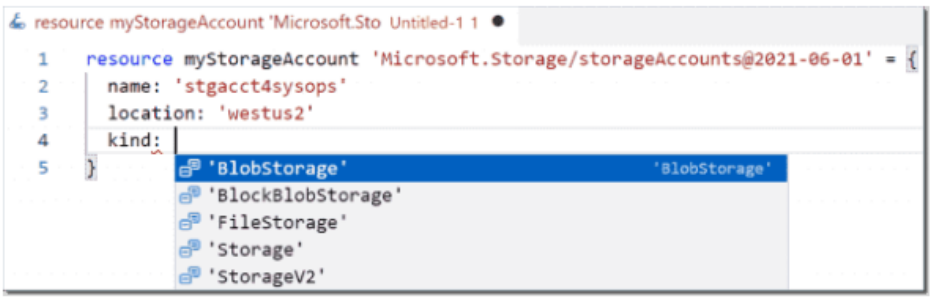
- To install it, go to the "Extensions" section in Visual Studio Code or search for it in the Visual Studio marketplace.
- Look for "bicep" and click "Install."



- Once it's installed, when you open a file that ends with ".bicep," you'll notice the language mode changes to Bicep.



When you have the Bicep extension added to Visual Studio Code, it becomes helpful. It gives you suggestions as you type, especially for resource properties in your Bicep templates. For instance, if you're defining a storage account, Visual Studio Code will suggest possible values for properties like "kind" as you write them in your code. This makes it easier to create accurate Bicep templates.



The Bicep extension provides autocomplete values for resource definition

Bicep Template Components

Like other languages used for infrastructure-as-code, Bicep templates consist of different parts and syntax for describing resources. The upcoming sections will introduce a few of these parts and features. To get started, make an empty file named "vnet.bicep." You'll use this file to write a template for deploying a virtual network.

Declaring Resources

The main objective of Bicep is to define resources, and these definitions include several important elements. Below is a fundamental structure for defining a resource, beginning with the keyword "resource":

```
resource <symbolic name> <resource type>@<api version> = {
  //Required properties
  name: <resource name>
  location: <Azure region>

  // Remaining resource-specific properties
  ...
}
```

Let's examine each component and understand its role in the declaration:

- Symbolic name:

The symbolic name serves as an identifier for the resource within the Bicep template. It's important to note that this name won't be the actual name of the deployed resource. The real resource name is specified later in the resource block. Symbolic names are case-sensitive.

- Resource type:

The resource type defines the specific kind of resource you intend to deploy. If you're familiar with ARM templates, these resource types will feel familiar. The format includes the resource provider followed by the resource name. Examples of resource types are Microsoft.Network/virtualNetworks or Microsoft.Storage/storageAccounts. To explore the full list of resource types, you can refer to the following:Azure template documentation

- API version:

The API version plays a role in determining the properties and features available when defining a resource. This property functions similarly to specifying an API version in an ARM template.

- Resource properties:

All resources require at least a name and a deployment location. The properties that are mandatory or optional are influenced by the combination of the resource type and API version.

In this example:

```
resource myVnet 'Microsoft.Network/virtualNetworks@2021-03-01' = {
  name: 'vnet-app-prod-westus2'
  location: 'westus2'
  properties: {
    addressSpace:{
      addressPrefixes:[
        '10.30.0.0/16'
      ]
    }
  }
}
```

- resource myVnet 'Microsoft.Network/virtualNetworks@2021-03-01'defines a virtual network resource with the symbolic name myVnet, utilizing the specified resource type (Microsoft.Network/virtualNetworks) and API version (2021-03-01).
- name: 'vnet-app-prod-westus2' sets the name of the deployed virtual network resource.
- location: 'westus2' specifies the deployment location as West US 2.
- properties encapsulate the resource-specific properties.
- addressSpace defines the address prefixes for the virtual network.
- addressPrefixes is an array that contains the specified address prefix ('10.30.0.0/16') for the virtual network.

By using these components, you can create a basic Bicep template for deploying a virtual network.

Parameters

Parameters are essential components in Bicep templates (as well as in other infrastructure-as-code languages like ARM templates). They allow you to make your templates more dynamic by accepting input values during deployment. These input values can be used to customize the behavior of your template, making it adaptable to various scenarios without requiring changes to the template code itself.

In the context of Bicep templates, parameters serve as placeholders for values that can be provided when you deploy the template. They provide a way to externalize configurable properties from the template logic, making your templates more reusable and easier to maintain.

To incorporate parameters into your Bicep template, consider the following example:

```
param environment string = 'dev'

param location string = resourceGroup().location

param allowedEnvironments string[] = ['prod', 'qa', 'dev']

resource myVnet 'Microsoft.Network/virtualNetworks@2021-02-01' = {
    name: 'vnet-${environment}-${location}'
    location: location
    properties: {
        addressSpace: {
            addressPrefixes: [
                '10.30.0.0/16'
            ]
        }
    }
}

output vnetName string = myVnet.name
```

- The param environment string = 'dev' line establishes a parameter named environment with a default value of 'dev'. This parameter allows you to specify the deployment environment, and 'dev' is the default if no value is provided.
- The param location string = resourceGroup().location line introduces a parameter named location with a default value derived from the resource group's location. If no location is specified during deployment, the template employs the resource group's location as the default.
- The param allowedEnvironments string[] = ['prod', 'qa', 'dev'] line establishes a parameter named allowedEnvironments as an array containing allowed environment values ('prod', 'qa', and 'dev').
- The name: 'vnet-${environment}-${location}' line in the resource myVnet declaration uses the environment and location parameters to construct the name of the virtual network.
- The output vnetName string = myVnet.name section defines an output named vnetName that captures the name of the deployed virtual network.

Incorporating parameters enhances the flexibility of your Bicep template, enabling customization for different deployment scenarios.

For additional information on the available Bicep functions familiarize yourself with the following resource.here.

Variables

Variables in Bicep templates serve to store and reuse values or expressions that are used multiple times throughout the template. They make it easier to manage complex or frequently used values by defining them once and referring to the variable name elsewhere in the template.

Defining Variables

To define variable, you must:

- Use the var keyword followed by the variable name.

• Use the equals sign (=) to assign a value or an expression to the variable.

For instance, if you're creating a Bicep template for deploying the network components of an application and other resources, you might want to create a consistent naming convention based on the application name, environment, and location.

ExampleLet's say you define a variable named appName using string interpolation to create this naming convention. This technique involves creating placeholders within a string that are replaced with actual values when the variable is used.

var appName = 'app-${environment}-${location}'

In this example, the appName variable is defined to create a base name for Azure resources. It combines the values passed through the environment and location parameters using string interpolation.

Usage of VariablesWith the parameters and the appName variable defined, you can now use these values to update your template. For instance, in your virtual network resource declaration, you can utilize the appName variable to create a meaningful name for your virtual network. It looks like this:

```
param environment string = 'dev'
param location string = resourceGroup().location

var appName = 'app-${environment}-${location}'

resource myVnet 'Microsoft.Network/virtualNetworks@2021-02-01' = {
  name: 'vnet-${appName}'
  location: location
  properties: {
    addressSpace: {
      addressPrefixes: [
        '10.30.0.0/16'
      ]
    }
  }
}

output vnetName string = myVnet.name
```

Deploying Bicep Templates

You can choose to use either Azure PowerShell or Azure CLI for the deployment. Here's how you can deploy a Bicep template using both methods:

• Using Azure CLI:

Assuming you have already authenticated with Azure using az login, you can use the following command to deploy a Bicep template:

```
resourceGroupName="rg-bicepdemo"
templateFile="path/to/your/template.bicep"

az deployment group create \
    --resource-group $resourceGroupName \
    --template-file $templateFile \
    --parameters environment=dev location=WestUS2
```

Replace "path/to/your/template.bicep" with the actual path to your Bicep template file.

• Using Azure PowerShell:

Assuming you have already authenticated with Azure using Connect-AzAccount, you can use the following command to deploy a Bicep template:

```
$resourceGroupName = "rg-bicepdemo"
$templateFile = "path/to/your/template.bicep"
$templateParameters = @{
    "environment" = "dev"
    "location" = "WestUS2"
}

New-AzResourceGroupDeployment -ResourceGroupName $resourceGroupName -TemplateFile $templateFile
                              -TemplateParameterObject $templateParameters
```

Summary

Conclusion

In this advanced module on Bicep, we delved into various sophisticated aspects of the Bicep language for Azure resource deployment. We explored the significance of resource types, the usage of parameters for dynamic configuration, and the power of variables for code reuse. This completed the content portion of this course. Time has now been set aside for you to finish your project.

References

Brown, Jeff. 4sysops. 2021. Azure Bicep: Getting Started guide. [Webpage] Retrieved from https://4sysops.com/archives/azure-bicep-getting-started-guide/

Microsoft Ignite. 2023. Install Bicep Tools. [Webpage] Retrieved from https://learn.microsoft.com/en-us/azure/azure-resource-manager/bicep/install

Microsoft Ignite. 2023. Quickstart: Create and deploy Bicep files using Visual Studio Code. [Webpage] Retrieved from https://learn.microsoft.com/en-us/azure/azure-resource-manager/bicep/quickstart-create-bicep-use-visual-studio-code?tabs=CLI

Yusubov, Elkhan. LinkedInLearning. 2021. Azure Bicep First Look. [Video] Retrieved from Learning Azure Bicep (linkedin.com)